Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mdmr functions

Synopsis

The functions prefixed with Mdmr make up the SureDotOCR module. The SureDotOCR module performs optical character recognition on dot-matrix text using feature-based technologies. You can quickly create fonts and string models and use them to perform robust read operations. SureDotOCR is invariant to changes in contrast, and offers considerable tolerance towards variations in angle. The module can also adapt to some variations in scale, aspect ratio, and perspective. SureDotOCR supports multiple grammar-like rules and font definitions in a single context, and provides its own font file format, allowing you to more easily manage your fonts.

Functions

MdmrAlloc
MdmrAllocResult
MdmrControl
MdmrControlFont
MdmrControlStringModel
MdmrDraw
MdmrExportFont
MdmrFree
MdmrGetResult
MdmrImportFont
MdmrInquire
MdmrInquireFont
MdmrInquireStringModel
MdmrName
MdmrPreprocess
MdmrRead
MdmrRestore
MdmrSave
MdmrStream

MdmrAlloc

Synopsis

Allocate a SureDotOCR context.

Syntax

```
AIL_ID MdmrAlloc(
    AIL_ID SysId,           //in
    AIL_INT64 ContextType,  //in
    AIL_INT64 ControlFlag,  //in
    AIL_ID *ContextDmrIdPtr //out
)
```

Description

This function allocates a SureDotOCR context on the specified system. A SureDotOCR context contains information required to use the Aurora Imaging Library SureDotOCR functions, which are prefixed with Mdmr; these functions allow you to read strings of dot-matrix text. Global settings, fonts, and string models are all held in a SureDotOCR context; use MdmrControl(), MdmrImportFont(), and MdmrControlStringModel() to set these up, respectively. Use MdmrRead() to perform the actual read operation. To read successfully, a SureDotOCR context must have at least one font and one string model.

A SureDotOCR context internally handles characters in Unicode.

When the SureDotOCR context is no longer required, release it using MdmrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the SureDotOCR context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the SureDotOCR context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
Aurora Imaging Library system identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of context to allocate. Set this parameter to the value below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of context to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DOT_MATRIX	Specifies to allocate an Aurora Imaging Library SureDotOCR context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextDmrIdPtr

Specifies the address of the variable in which to write the SureDotOCR context identifier or specifies the data type that the function should use to return the SureDotOCR context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated SureDotOCR context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated SureDotOCR context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DMR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the SureDotOCR context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated SureDot OCR context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the SureDotOCR context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DMR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrAllocResult

Synopsis

Allocate a SureDotOCR result buffer.

Syntax

```
AIL_ID MdmrAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 ResultType,  //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ResultDmrIdPtr //out
)
```

Description

This function allocates a SureDotOCR result buffer, on the specified system, to store results obtained from an MdmrRead() operation.

When the SureDotOCR result buffer is no longer required, release it using MdmrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the SureDotOCR result buffer's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the SureDotOCR result buffer. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result buffer to allocate. Set this parameter to the value below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result buffer to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DOT_MATRIX	Specifies to allocate a SureDotOCR result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ResultDmrIdPtr

Specifies the address of the variable in which to write the SureDotOCR result buffer identifier or specifies the data type that the function should use to return the SureDotOCR result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated SureDotOCR result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated SureDotOCR result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DMR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the SureDotOCR result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated SureDot OCR result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the SureDotOCR result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DMR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrControl

Synopsis

Control a SureDotOCR context or result buffer.

Syntax

```
void MdmrControl(
    AIL_ID ContextOrResultDmrId, //out
    AIL_INT64 ControlType,       //in
    AIL_DOUBLE ControlValue      //in
)
```

Description

This function allows you to control a SureDotOCR context or a SureDotOCR result buffer. This includes adding or deleting a font or string model from a context, or removing all results from a result buffer. To inquire about a context, call MdmrInquire(). To get results from a result buffer, call MdmrGetResult().

To control the fonts and string models held by a SureDotOCR context, call MdmrControlFont() and MdmrControlStringModel(), respectively.

For a successful read operation, you must specify an explicit value for the control types that set the diameter of the dots in the string's characters (M_DOT_DIAMETER). You must specify the dimensions of the dot-matrix text block using M_TEXT_BLOCK_HEIGHT and M_TEXT_BLOCK_WIDTH; the dot-matrix text block is the area in the image buffer (or in the buffer's region of interest) that includes all dot-matrix text.

You must preprocess the SureDotOCR context after you have finished adjusting context settings and before calling MdmrRead(). To know if a context needs to be preprocessed, call MdmrInquire() with M_PREPROCESSED.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MdmrControl()** internally calls the *AIL_DOUBLE* version of this function (MdmrControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MdmrControlInt64()).

Parameters

ContextOrResultDmrId

Specifies the identifier of the SureDotOCR context or result buffer to control. The context must have been previously allocated on the system using MdmrAlloc(). The result buffer must have been previously allocated on the system using MdmrAllocResult().

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For controlling a SureDotOCR context setting
- For adding fonts or string models to, or deleting fonts or string models from, a SureDotOCR context
- For controlling a SureDotOCR result buffer

The following ControlType and corresponding ControlValue parameter settings are used to control a setting of the SureDotOCR context specified with the ContextOrResultDmrId parameter. SureDotOCR can only read strings that adhere to these settings (they apply to all strings in the context). If there are strings that require different settings, use a separate context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a SureDotOCR context setting																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_DOT_DIAMETER	Sets the diameter of the dots that make up the characters in the strings. INQ															

Value > 0.0	Specifies the diameter, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_DIAMETER_SPREAD	Sets the supported spread of the dots. This essentially sets the tolerance for the size of the dot diameter, as a value centered around the specified nominal dot diameter (M_DOT_DIAMETER). For example, if M_DOT_DIAMETER is set to 4.0 and this control type is set to 2.0, Aurora Imaging Library will read all dots between the size of 3.0 and 5.0. This control type is ignored if M_DOT_DIAMETER_SPREAD_MODE is set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the tolerance for the size of the dot diameter, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_DIAMETER_SPREAD_MODE	Sets whether the dot diameter can vary within a specified range of possible diameters specified using M_DOT_DIAMETER.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the M_DOT_DIAMETER_SPREAD is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that M_DOT_DIAMETER_SPREAD is used during a read operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_DIAMETER_STEP	Sets the increment between steps that will be used when checking for dots in the range of supported dot diameters, set using M_DOT_DIAMETER_SPREAD. For example, if M_DOT_DIAMETER is set to 1.0, M_DOT_DIAMETER_SPREAD is set to 2.0, and this control type is set to 0.2, Aurora Imaging Library will check 1.2, 1.4, 1.6 and so on. This control type is ignored if M_DOT_DIAMETER_SPREAD_MODE is set to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the step, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE	Sets the foreground value of the characters in the strings. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FOREGROUND_BLACK.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that strings with characters darker than the background will be read. For example, black characters on a white surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that strings with characters lighter than the background will be read. For example, white characters on a black surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_ANGLE	Sets the italic angle of all the characters in the strings. The angle must be provided counter clockwise and is relative a line perpendicular to the angle of the string. M_ITALIC_ANGLE only has an effect if M_ITALIC_ANGLE_MODE is set to M_ANGLE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
-90.0 <= *Value* <= 90.0	Specifies the italic angle, in degrees. A positive angle means that the characters are leaning counter-clockwise (left leaning); a negative angle means that the characters are leaning clockwise (right leaning).	c		e	f	g	a		i		j	k	l	m	n	b

M_ITALIC_ANGLE_MODE	Sets whether the italic angle is detected automatically or explicitly specified. M_ITALIC_ANGLE_MODE must be set to M_DEFAULT if M_STRING_ANGLE_MODE is set to M_DEFAULT; otherwise, MdmrPreprocess() will generate an error when called. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE	Specifies to use the italic angle specified with M_ITALIC_ANGLE. A string will only be read if its characters have the specified italic angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the italic angle is detected automatically. M_ITALIC_ANGLE is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_PITCH	Sets the distance between successive dot centers in the italic angle direction. M_ITALIC_PITCH only has an effect if M_ITALIC_PITCH_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the pitch, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_PITCH_MODE	Sets whether the distance between successive dot centers, in the italic angle direction, is detected automatically or explicitly specified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the distance between dot centers in the italic angle direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the value set with M_ITALIC_PITCH. A string will only be read if its characters have the specified italic pitch.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY	Sets the maximum pixel intensity of a character's dots. M_MAX_INTENSITY only has an effect if M_MAX_INTENSITY_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 255.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 255.0	Specifies the maximum pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY_MODE	Sets how to establish the maximum pixel intensity of a character's dots. For a dot to be part of a character, its pixel intensity must be less than or equal to the maximum intensity. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the maximum pixel intensity of a character's dots. By default, SureDotOCR considers dots with any pixel intensity less than or equal to 255.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the maximum pixel intensity value set with M_MAX_INTENSITY.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_CONTRAST	Sets the minimum contrast between a character and its background. M_MIN_CONTRAST only has an effect if M_MIN_CONTRAST_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 15.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 255.0	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_CONTRAST_MODE	Sets how to establish the minimum contrast (difference in pixel intensity) between a character (foreground) and its background. For a character to be read, its contrast must be greater than or equal to the minimum contrast. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the minimum contrast of characters. Regardless of the automatically calculated value, SureDotOCR will not read characters with a contrast less than 15.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the minimum contrast value set with M_MIN_CONTRAST.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_INTENSITY	Sets the minimum pixel intensity of a character's dots. M_MIN_INTENSITY only has an effect if M_MIN_INTENSITY_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 255.0	Specifies the minimum pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_INTENSITY_MODE	Sets how to establish the minimum pixel intensity of a character's dots. For a dot to be part of a character, its intensity must be greater than or equal to the minimum intensity. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the minimum pixel intensity of character dots. By default, SureDotOCR considers dots with any pixel intensity greater than or equal to 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the minimum pixel intensity value set with M_MIN_INTENSITY.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MAX	Sets the value with which to determine the maximum width of a space character. This value is interpreted according to M_SPACE_SIZE_MAX_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum width value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MAX_MODE	Sets how to determine the maximum width of a space character. If the actual distance between two characters is greater than the maximum width, the read operation considers them part of separate strings. M_SPACE_SIZE_MAX_MODE indicates how to interpret M_SPACE_SIZE_MAX. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_CHAR_WIDTH_FACTOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_WIDTH_FACTOR	Specifies that the maximum width of a space character is determined by multiplying the M_SPACE_SIZE_MAX value by the string's maximum character width, in pixels. For example, if M_SPACE_SIZE_MAX is set to 3 (M_DEFAULT), the maximum space character width is equivalent to three times the string's maximum character width.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies no maximum space width. This is equivalent to allowing an infinite amount of maximum space. In this case, no amount of space between aligned characters can cause them to be part of separate strings. This does not apply to characters on separate lines or severely misaligned characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MIN	Sets the value with which to determine the minimum width of a space character. This value is interpreted according to M_SPACE_SIZE_MIN_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the minimum width value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MIN_MODE		Sets how to determine the minimum width of a space character. If the actual distance between two characters is less than the minimum width, SureDotOCR does not consider there to be a space. M_SPACE_SIZE_MIN_MODE indicates how to interpret M_SPACE_SIZE_MIN. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_CHAR_WIDTH_FACTOR.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CHAR_WIDTH_FACTOR	Specifies that the minimum width of a space character is determined by multiplying the M_SPACE_SIZE_MIN value by the string's maximum character width, in pixels. For example, if M_SPACE_SIZE_MIN is set to 1 (M_DEFAULT), the minimum space character width is equivalent to the string's maximum character width.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies no minimum space width. This is equivalent to allowing an infinite amount of minimum space. In this case, no amount of space between characters in a string can cause SureDotOCR to recognize an actual space.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE		Sets at which angle to read a string, with respect to the coordinate system specified with M_STRING_ANGLE_INPUT_UNITS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_REGION	Specifies that the angle is set to the angle of the target image's ROI. If this value is chosen but no ROI has been defined, the angle is set to 0.0° in the coordinate system specified with M_STRING_ANGLE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies a fixed angle at which to read a string, in degrees. This is interpreted with respect to the X-axis of the coordinate system specified with M_STRING_ANGLE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_INPUT_UNITS		Sets the coordinate system to use for M_STRING_ANGLE. This control type is ignored when M_STRING_ANGLE is set to M_ACCORDING_TO_REGION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_MODE		Sets whether the string angle is detected automatically or explicitly specified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANGLE	Specifies to use the angle specified with M_STRING_ANGLE; the string will be read from left to right with the characters facing upward along the specified angle. A string will only be read if it has the specified angle.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies that the string angle is detected automatically; M_STRING_ANGLE is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORIENTATION	Specifies that the string will be read from left to right with the characters facing upward, or from right to left with the character facing downward. This is useful in cases where the orientation of a string is rotated 180 degrees, and this rotation does not affect the acceptability of a product (for example, if the product is placed backwards at an inspection point but is still a good part).	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_PARTIAL_CHAR_INVALID	Sets the single character string to replace invalid or unrecognized characters if M_STRING_PARTIAL_MODE is enabled. The default value is "?". Note that the specified character is validated during the preprocess operation to confirm that the selected character does not appear in any font in the context.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("?") 1	Specifies the default value as "?". > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("?")).	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*SingleCharacterString*") 1	Specifies the single character string to replace the invalid or unrecognized characters. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*SingleCharacterString*")).	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PARTIAL_MODE	Sets how partial strings are returned during the reading process.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that only complete strings will be returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the best match for a partial string will be returned; unrecognized characters in the string will be replaced with M_STRING_PARTIAL_CHAR_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PITCH	Sets the distance between successive dot centers, in the string angle direction. M_STRING_PITCH only has an effect if M_STRING_PITCH_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the pitch, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PITCH_MODE	Sets whether the distance between successive dot centers, in the string angle direction, is detected automatically or explicitly specified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the distance between dot centers in the string angle direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the value set with M_STRING_PITCH. A string will only be read if it has the specified string pitch.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_BLOCK_HEIGHT	Sets the height of the text block. This is the area that encloses all dot-matrix text in the image buffer (or in the buffer's region of interest). The height is oriented along the vertical axis that is perpendicular to the base of the string(s) to read in the text block. M_TEXT_BLOCK_HEIGHT only has an effect if M_TEXT_BLOCK_SIZE_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_BLOCK_SIZE_MODE	Sets how to establish the size of the text block. This is the area that encloses all dot-matrix text in the image buffer (or in the buffer's region of interest). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the size of the rectangular block is defined by the M_TEXT_BLOCK_HEIGHT and M_TEXT_BLOCK_WIDTH settings.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEXT_BLOCK_WIDTH	Sets the width of the text block. This is the area that encloses all dot-matrix text in the image buffer (or in the buffer's region of interest). The width is oriented along the axis that represents the base of the string(s) to read in the text block. M_TEXT_BLOCK_WIDTH only has an effect if M_TEXT_BLOCK_SIZE_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT	Sets the maximum read time for MdmrRead(). No results are returned if the read operation times out. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2000.0 msec.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies an infinite amount of read time.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum read time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

The following ControlType and corresponding ControlValue parameter settings are used to add fonts or string models to, or delete fonts or string models from, the SureDotOCR context specified with the ContextOrResultDmrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For adding fonts or string models to, or deleting fonts or string models from, a SureDotOCR context

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_ADD	Adds an empty font to the context. To add characters to a font, use MdmrImportFont(). You can also add characters to a font, or modify its settings, using MdmrControlFont(). The maximum number of fonts in a context is 255. To inquire the number of fonts in the context, call MdmrInquire() with M_NUMBER_OF_FONTS. When you add an empty font, the dimensions of its dot-matrix template are initially set to 5 columns and 7 rows. If the font is empty, you can change these values by calling MdmrControlFont() with M_FONT_SIZE_COLUMNS and M_FONT_SIZE_ROWS.															
M_DEFAULT	Same as M_NEW_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEW_LABEL	Specifies to add a font with an automatically assigned label. To inquire about the label, use MdmrInquireFont() with M_FONT_LABEL_VALUE. To change the label, use MdmrControlFont() with M_FONT_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies to add a font with the indicated label, as an *integer*. The label must be unique among all font labels in the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_DELETE	Deletes one or all fonts from the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_INDEX(AIL_INT IndexValue)	Specifies to delete the font by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_FONTS — Specifies the index of the font to delete.	c		e	f	g	a		i		j	k	l	m	n	b

M_FONT_LABEL(AIL_INT LabelValue)	Specifies to delete the font by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Set this parameter to the following:															
	Value > 0 — Specifies the label of the font to delete.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to delete all fonts.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ADD	Adds an empty string model to a context. To add character constraints to the string model, or to modify its settings, use MdmrControlStringModel(). The maximum number of string models in a context is 255. To inquire the number of string models in a context, call MdmrInquire() with M_NUMBER_OF_STRING_MODELS.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NEW_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEW_LABEL	Specifies to add a string model with an automatically assigned label. To inquire about the label, use MdmrInquireStringModel() with M_STRING_LABEL_VALUE. To change the label, use MdmrControlStringModel() with M_STRING_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies to add a string model with the indicated label, as an *integer*. The label must be unique among all string model labels in the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_DELETE	Deletes one or all string models from a context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to delete the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS — Specifies the index of the string model to delete.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies to delete the string model by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Set this parameter to the following:															
	Value > 0 — Specifies the label of the string model to delete.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to delete all string models.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the SureDotOCR result buffer specified with the ContextOrResultDmrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a SureDotOCR result buffer

ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_RESET	Removes all results from the SureDotOCR result buffer. This does not delete the result buffer identifier, as is the case with MdmrFree().															
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MstrGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_READ	Stops the current read operation. You can only perform this operation from another thread of higher priority. No results are returned if you stop the read operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MdmrControlInt64 (AIL_ID ContextOrResultDmrId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResultDmrId

See ContextOrResultDmrId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MdmrControlDouble (AIL_ID ContextOrResultDmrId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultDmrId

See ContextOrResultDmrId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrControlFont

Synopsis

Control a global setting of a font in a SureDotOCR context, or control a character in a font.

Syntax

```
void MdmrControlFont(
    AIL_ID ContextDmrId,            //out
    AIL_INT64 FontLabelOrIndex,     //in
    AIL_INT64 CharIndex,            //in
    AIL_CONST_TEXT_PTR CharNamePtr, //in
    AIL_INT64 ControlType,          //in
    AIL_DOUBLE ControlValue,        //in
    const void *ControlValuePtr     //in
)
```

Description

This function allows you to control a global setting of a font in a SureDotOCR context, or to control a character in a font. This includes adding characters to, and deleting characters from, a font. To add a font to a SureDotOCR context, use MdmrControl() with M_FONT_ADD (this adds an empty font). For a successful read operation, the context must have at least one font with one character. The characters held by a font are represented by a dot-matrix. String models use the dot-matrix character representations in the font to define the text to read.

Adding or modifying characters with this function requires specifying their dot-matrix representation in an array. You can also add or modify characters by calling MdmrImportFont() with a SureDotOCR font file (more commonly done). To inquire about fonts, use MdmrInquireFont().

You must preprocess the SureDotOCR context after modifying its fonts and before calling MdmrRead(). To know if a context needs to be preprocessed, call MdmrInquire() with M_PREPROCESSED.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MdmrControlFont()** internally calls the *AIL_DOUBLE* version of this function (MdmrControlFontDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MdmrControlFontInt64()).

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context that contains the fonts to control. The context must have been previously allocated on the system using MdmrAlloc().

FontLabelOrIndex

Specifies the font (one or all) to control. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the font																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_INDEX(AIL_INT IndexValue)	Specifies to control the font by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the font.															
	0 <= *Value* < M_NUMBER_OF_FONTS Specifies the index of the font to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL(AIL_INT LabelValue)	Specifies to control the font by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Specifies the label of the font.															

	0 < *Value* < 2097152	Specifies the label of the font to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to control all fonts.		c		e	f	g	a		i		j	k	l	m	n	b

CharIndex

Specifies the index of a character in the font, if required. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of a character																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the index of a character is not required.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies all characters. Only specify this value for M_CHAR_DELETE.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of a character.	c		e	f	g	a		i		j	k	l	m	n	b

CharNamePtr

Specifies the name of a character in the font, if required. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the character																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that the name of the character is not required.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*CharName*") 1	Specifies the name of the character. For example, 'A' would be the name that identifies the first uppercase letter of the English alphabet. When using the CharNamePtr parameter to specify the name of a character, the CharIndex parameter must be set to M_DEFAULT. When working with data in an ASCII format, you can specify Unicode character names; these names must be in hexadecimal format and begin with "\x". This is required for specifying character names beyond the Basic Latin range. For example, Basic Latin does not include the smiley face character; to specify it, use "\x263A". When working in a Unicode environment, you can directly specify the smiley face as the name of the character. Character names must be unique among all character names in the font. Specifying the name of a character that already exists causes an error. To avoid unpredictable results, character names should match the dot-matrix that they represent.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlValuePtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]
- AIL_CONST_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- M_NULL

Specifies the address which contains more information about the setting's new value.

Set this parameter to **M_NULL** if not used. If you use this parameter, the ControlValue parameter must be M_DEFAULT.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType**, **ControlValue**, and **ControlValuePtr** parameters.

- For controlling a global setting of a font
- For controlling a font's characters

The following ControlType and corresponding ControlValue and ControlValuePtr parameter settings are used to control a global setting of a font. Unless otherwise specified, set the FontLabelOrIndex parameter to the label or index of a font (one or all), set the CharIndex parameter to M_DEFAULT, and set the CharNamePtr parameter to M_NULL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a global setting of a font																	
ControlType / ControlValue / ControlValuePtr	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_LABEL_VALUE	Modifies the label value of the font. Set the FontLabelOrIndex parameter to a specific font. Specifying all fonts causes an error.															
ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
0 < *Value* < 2097152	Specifies the font's new label value, as an *integer*. The label must be unique among all font labels in the context.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_SIZE_COLUMNS	Sets the number of columns in the font's dot-matrix template. Only specify this value for an empty font (no characters). The dot-matrix template defines the grid within which to represent every character in the font. Fonts are typically added with MdmrImportFont(). In this case, the dimensions of the characters' dot-matrix in the font file (MDMRF) represent the dimensions of the dot-matrix template of the font. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 5 columns.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of columns, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_SIZE_ROWS	Sets the number of rows in the font's dot-matrix template. Only specify this value for an empty font (no characters). The dot-matrix template defines the grid within which to represent every character in the font. Fonts are typically added with MdmrImportFont(). In this case, the dimensions of the characters' dot-matrix in the font file (MDMRF) represent the dimensions of the dot-matrix template of the font. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 7 rows.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of rows, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue and ControlValuePtr parameter settings are used to control a font's characters. Unless otherwise specified, set the FontLabelOrIndex parameter to the label or index of a font (one or all), and set either the CharIndex parameter to a character index or the CharNamePtr parameter to the name of a character.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a font's characters																
ControlType ControlValue ControlValuePtr	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CHAR_ADD	Adds a character to the font. To add multiple characters, call this function multiple times. SureDotOCR assigns an index to each character, starting with 0. Use the CharNamePtr parameter to specify the name of the character and set the CharIndex parameter to M_DEFAULT. Fonts should only contain characters you want to read.															
ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]* Specifies the address of an array containing the dot-matrix of the character to add to the font. You must define the dot-matrix in the array, one row contiguously after the other; use "FF" for dots that represent the character; otherwise use "00". The dot-matrix of the font's characters should be identical to the dot-matrix of the characters in the strings to read. You must organize the character data in the specified array according to the dimensions of the font's dot-matrix template. To inquire the dimensions, call MdmrInquireFont() with M_FONT_SIZE_COLUMNS and M_FONT_SIZE_ROWS. The dot-matrix of all characters in a font must have these dimensions. The dots that make up the dot-matrix are specific to each character.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_DELETE	Deletes characters in a font. To delete a specific character in a font, set the CharIndex parameter to the index of the character to delete, or set the CharNamePtr parameter to the name of the character to delete. To delete all characters in a font, set the CharIndex parameter to M_ALL and the CharNamePtr parameter to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NAME	Renames a character in a font.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*CharName*") 1	Specifies the character's new name. To indicate the character to rename, use either the CharIndex parameter or the CharNamePtr parameter. Character names must be unique among all character names in the font. To avoid unpredictable results, character names should match the dot-matrix that represents them.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_TEMPLATE	Redefines the dot-matrix of a character in a font. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	ControlValue	Specifies the following.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]* Specifies the address of an array containing the character's new dot matrix. This character data must adhere to the same specifications as M_CHAR_ADD. To indicate the character to redefine, use either the CharIndex parameter or the CharNamePtr parameter.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Type-specific versions of the function when using a C compiler under 64-bit

```
void MdmrControlFontInt64 (AIL_ID ContextDmrId, AIL_INT64 FontLabelOrIndex, AIL_INT64 CharIndex, AIL_CONST_TEXT_PTR CharNamePtr, AIL_INT64 ControlType, AIL_INT64 ControlValue, const void *ControlValuePtr)
```

Details

Parameters

ContextDmrId

See ContextDmrId of the main function for a description.

FontLabelOrIndex

See FontLabelOrIndex of the main function for a description.

CharIndex

See CharIndex of the main function for a description.

CharNamePtr

See CharNamePtr of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

ControlValuePtr

See ControlValuePtr of the main function for a description.

```
void MdmrControlFontDouble (AIL_ID ContextDmrId, AIL_INT64 FontLabelOrIndex, AIL_INT64 CharIndex, AIL_CONST_TEXT_PTR CharNamePtr, AIL_INT64 ControlType, AIL_DOUBLE ControlValue, const void *ControlValuePtr)
```

Details

Parameters

ContextDmrId

See ContextDmrId of the main function for a description.

FontLabelOrIndex

See FontLabelOrIndex of the main function for a description.

CharIndex

See CharIndex of the main function for a description.

CharNamePtr

See CharNamePtr of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

ControlValuePtr

See ControlValuePtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrControlStringModel

Synopsis

Control a global setting of a SureDotOCR string model, or control a position (constraint) in a string model.

Syntax

```
void MdmrControlStringModel(
    AIL_ID ContextDmrId,               //out
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 Position,                //in
    AIL_INT64 ControlType,             //in
    AIL_DOUBLE ControlValue1,          //in
    AIL_DOUBLE ControlValue2,          //in
    const void *ControlValuePtr        //in
)
```

Description

This function allows you to control a global setting of a SureDotOCR string model, or to control a position in a string model. This includes setting constraints for the different positions. Constraints restrict the characters permitted to be read, according to a character type and font. String models, along with their constraints, represent the dot-matrix strings to read from a target image. Initially, any character from any font can be read at every string model position.

String models are held in a SureDotOCR context. To add string models to a context, use MdmrControl(). To inquire about string models, use MdmrInquireStringModel().

You can set default constraints for all the positions in the string model, and override the constraints for specific positions. If you override the constraints for a specific position, the position is said to be explicitly constrained; otherwise, it is said to be implicitly constrained. To set the default constraints for the positions in the string model, call this function and pass M_DEFAULT to the Position parameter; to set explicit constraints for a position, pass M_POSITION_IN_STRING(n) instead, where *n* is the position to constrain.

If you previously constrained a position, you can further constrain it by specifying its exact position in the string model, or by indicating the order in which it was originally constrained. For example, if you explicitly constrained positions 14 and 22, you can further constrain position 22 using either M_POSITION_IN_STRING(n), where *n* is 22, or M_POSITION_CONSTRAINED_ORDER(n), where *n* is 1.

Once you have explicitly constrained a position, it is considered to be explicitly constrained until you call this function with M_RESET_POSITION_TO_IMPLICIT_CONSTRAINTS for that position, even if you have manually set the position's constraints back to the defaults of the string model. Note that the constrained order of a position changes if you reset previously constrained positions to be implicitly constrained (M_RESET_POSITION_TO_IMPLICIT_CONSTRAINTS).

For a successful read operation, you must specify an explicit value for the global string model controls that set the maximum and minimum number of characters in the string (M_STRING_SIZE_MIN and M_STRING_SIZE_MAX, or M_STRING_SIZE_MIN_MAX).

You must preprocess the SureDotOCR context after you have finished modifying its string models and before calling MdmrRead(). To know if a context needs to be preprocessed, call MdmrInquire() with M_PREPROCESSED.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MdmrControlStringModel()** internally calls the *AIL_DOUBLE* version of this function (MdmrControlStringModelDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MdmrControlStringModelInt64()).

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context that contains the string model to control. The context must have been previously allocated on the system using MdmrAlloc().

StringModelLabelOrIndex

Specifies the string model (one or all) to control. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to control the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	IndexValue Specifies the index of the string model.														
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies to control the string model by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	LabelValue Specifies the label of the string model.														
	0 < *Value* < 2097152 Specifies the label of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to control all string models.	c		e	f	g	a		i		j	k	l	m	n	b

Position

Specifies how to control the string model. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to control to the string model															
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to control a global setting of a string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_CONSTRAINED_ORDER(AIL_INT OrderValue)	Specifies to control an explicitly constrained position in the string model by indicating the order in which the position was explicitly constrained.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	OrderValue Specifies the order in which the position was explicitly constrained.														
	0 <= *Value* <= 255 Specifies the order in which the position was explicitly constrained. For example, if you explicitly constrained positions 19, 5, and 24, their respective constrained order values are 0, 1, and 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_IN_STRING(AIL_INT PositionValue)	Specifies to control a position in the string model.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	PositionValue Specifies the position in the string model.														
	0 <= *Value* <= 255 Specifies the position in the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_CONSTRAINED_POSITIONS	Specifies to control all explicitly constrained positions in the string model.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue1

Specifies the required value for the control.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlValue2

Specifies the second required value for the control.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlValuePtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_CONST_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- M_NULL

Specifies the address which contains more information about the setting's new value.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType**, **ControlValue1**, **ControlValue2**, and **ControlValuePtr** parameters.

- For controlling a global setting of a string model
- For setting constraints to the different positions in the string model
- For generally controlling the different positions in the string model

The following ControlType and corresponding ControlValue1, ControlValue2, and ControlValuePtr parameter settings are used to control a global setting of a string model. In this case, set the StringModelLabelOrIndex parameter to the label or index of a string model (one or all, unless otherwise specified), and set the Position parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a global setting of a string model																	
ControlType ControlValue1 ControlValue2 ControlValuePtr	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CHAR_ACCEPTANCE	Sets the acceptance level for the score of the string's characters. This score quantifies the similarity between the characters in the target string and the corresponding characters in the font that the string model uses. For the string to be read, the score of every character must be greater than or equal to the specified character acceptance. To retrieve character scores, call MdmrGetResult() with M_CHAR_SCORE. INQ															
ControlValue1	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the acceptance level for the character score, as a percentage. 100.0% indicates that the character in the target string perfectly resembles the corresponding character in the font (perfection is generally hard to obtain).	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b

ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ACCEPTANCE	Sets the acceptance level for the string's score. This score quantifies the similarity between the target string and the string model. For the string to be read, its score must be greater than or equal to the specified acceptance. The string's score is the average score of all its characters. Even if the string's score is above the string's acceptance, the string might not be read if one or more if its characters is below the character's acceptance (M_CHAR_ACCEPTANCE). In this case, consider lowering the character's acceptance. To retrieve the score of the string or its characters, call MdmrGetResult() with M_STRING_SCORE or M_CHAR_SCORE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the acceptance level for the string's score, as a percentage. 100.0% indicates that the string in the target image perfectly resembles the string model (perfection is generally hard to obtain).	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CERTAINTY	Sets the certainty level for the string's score. This score quantifies the similarity between the target string and the string model. If the string's score is equal to or above the certainty, the string is immediately read as a valid string result, without processing the rest of the target image for strings with higher scores (provided the specified number of strings to read has been read). The string's score is the average score of all its characters. To retrieve the score of the string or its characters, call MdmrGetResult() with M_STRING_SCORE or M_CHAR_SCORE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 70.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the certainty level for the string score, as a percentage. 100.0% indicates that the string in the target image perfectly resembles the string model (perfection is generally hard to obtain).	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL_VALUE	Modifies the label value of the string model. The StringModelLabelOrIndex parameter must specify a specific string model (not all).	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies the string model's new label value. The label value must be greater than 0 and it must be unique among all string models in the context.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_RANK	Sets the order in which to read a string, relative to the other strings to read. Strings with lower ranks are read before those with higher ones. Do not modify rank unless your context has multiple string models. SureDotOCR reads strings from left to right and from top to bottom, relative to the angle of the string's location. If a string model's rank is 0 (M_DEFAULT), SureDotOCR tries to read that string first; accounting for angle, it will be the top left-most string. If multiple string models have identical ranks, SureDotOCR reads just one string, depending on which is more similar to its string model. When specifying ranks, you must include rank 0, and you must not skip successive values. For example, you will get an error if a context has four string models with their ranks set to [1, 2, 3, 4] or [0, 1, 2, 4]. Strings with different ranks must be on separate lines. SureDotOCR uses the rank of the string models in a context to establish the required number of strings to read for that context: *TotalNumberOfStringsToRead* = *HighestRankValue* + 1. For example, if you have four string models with ranks set to [0, 0, 1, 2], SureDotOCR must read three strings. If 3 strings cannot be read, no results are returned. Since the default rank for every string model is 0, SureDotOCR will try to read 1 string, regardless of how many string models are in your context, unless you modify rank. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the order in which to read a string. Explicitly indicating rank can make the read operation faster and more accurate.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX	Sets the maximum number of characters in the string. This number includes spaces, if they correspond to a space permitted character constraint (M_SPACE). If this number is not equal to M_STRING_SIZE_MIN, use MdmrGetResult() with M_SKIPPED_POSITIONS to see which characters in the model were skipped. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies the maximum number of characters. This value must be equal to or greater than the minimum number of characters (M_STRING_SIZE_MIN).	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 256	Specifies the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN	Sets the minimum number of characters in the string. This number includes spaces, if they correspond to a space permitted character constraint (M_SPACE). If this number is not equal to M_STRING_SIZE_MAX, use MdmrGetResult() with M_SKIPPED_POSITIONS to see which characters in the model were skipped. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies the minimum number of characters. This value must be equal to or less than the maximum number of characters (M_STRING_SIZE_MAX).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 256	Specifies the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_SIZE_MIN_MAX	Sets the minimum and maximum number of characters in the string. These numbers include spaces, if they correspond to a space permitted character constraint (M_SPACE). If min and max are not equal, use MdmrGetResult() with M_SKIPPED_POSITIONS to see which characters in the model were skipped. M_STRING_SIZE_MIN_MAX is the same as calling this function twice, the first with M_STRING_SIZE_MIN and the second with M_STRING_SIZE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies the minimum number of characters. This value must be equal to or less than the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 256	Specifies the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Specifies the maximum number of characters. This value must be equal to or greater than the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 256	Specifies the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue1, ControlValue2, and ControlValuePtr parameter settings are used to set constraints for the different positions in the string model. In this case, set the StringModelLabelOrIndex parameter to the label or index of a string model (one or all).

To set the default constraints for the positions in the string model, set the Position parameter to M_DEFAULT, unless otherwise specified. To set explicit constraints for a position, set the Position parameter to the required position (one or all). Note that positions have zero-based indexing.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting constraints to the different positions in the string model																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue1																
ControlValue2																
ControlValuePtr																
M_ADD_PERMITTED_CHARS_ENTRY	Sets a constraint based on permitted characters. This restricts the characters that can be read, according to a font (ControlValue1) and character type (ControlValue2). For example, you can specify that at position 0, in string model 0, you will only permit the letter 'O', as represented in font *OhZero*, to be read. Every position in every string model can initially read any character from any font in the context, unless you specify otherwise. Spaces can also be read, depending on the space controls, which you can specify by calling MdmrControl() with M_SPACE_SIZE_MAX and M_SPACE_SIZE_MIN. Unless otherwise specified, permitted character constraints are cumulative. Each time you specify permitted characters, they are added to the previously specified permitted characters, if they exist.															
ControlValue1	Specifies the font that contains the representation of the character to read. The font must be in the specified context.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_INDEX(AIL_INT IndexValue)	Specifies the font by indicating its index, or specifies any font. When you specify a font's index, SureDotOCR internally refers to the font by its corresponding label. If the font's index subsequently changes, which can happen when other fonts are added or deleted, the font you referred to remains the same.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the font.															

			M_ANY	Specifies that the character can come from any font in the context.	c		e	f	g	a		i		j	k	l	m	n	b
			0 <= *Value* < M_NUMBER_OF_FONTS	Specifies the index of the font from which the character must come.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FONT_LABEL(AIL_INT LabelValue)	Specifies the font by indicating its label, or specifies any font.		c		e	f	g	a		i		j	k	l	m	n	b
			Parameters																
			LabelValue INQ Specifies the label of the font.																
			M_ANY	Specifies that the character can come from any font in the context.	c		e	f	g	a		i		j	k	l	m	n	b
			0 < *Value* < 2097151	Specifies the label of the font from which the character must come.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValue2		Specifies the type of character to read, relative to the specified font, unless otherwise specified. For example, to read digit type characters from *MySuperExcellentFont*, set this parameter to M_DIGITS, and set the ControlValue1 parameter to the index or label of *MySuperExcellentFont*. INQ		c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies to read all characters present in the font.		c		e	f	g	a		i		j	k	l	m	n	b
		M_CHAR_LIST	Specifies to read an explicit list of characters. To specify them, use the ControlValuePtr parameter.		c		e	f	g	a		i		j	k	l	m	n	b
		M_DIGITS	Specifies to read characters '0' to '9'.		c		e	f	g	a		i		j	k	l	m	n	b
		M_LETTERS	Specifies to read characters 'A' to 'Z' and 'a' to 'z'.		c		e	f	g	a		i		j	k	l	m	n	b
		M_LETTERS_LOWERCASE	Specifies to read characters 'a' to 'z'.		c		e	f	g	a		i		j	k	l	m	n	b
		M_LETTERS_UPPERCASE	Specifies to read characters 'A' to 'Z'.		c		e	f	g	a		i		j	k	l	m	n	b
		M_SPACE	Specifies to read a space. Unlike other permitted characters, fonts cannot represent a space. To establish it, call MdmrControl() with M_SPACE_SIZE_MAX and M_SPACE_SIZE_MIN. When specifying M_SPACE, set the ControlValue1 parameter to the index or label of any font (M_ANY). To use M_SPACE, the Position parameter must be set to a specific position that is not the first or last position in a string model. That position cannot have another character constraint nor can you have consecutive positions with a space constraint. If these conditions are not followed, you will get an error. Reading an M_SPACE character does not necessarily mean that the corresponding space position in the target string is blank. SureDotOCR attempts to read the best string possible, and can use the space constraint as an area to ignore, even if that area is not empty. For example, if you have an icon at a specific position within a target string, you can specify a space at that position.		c		e	f	g	a		i		j	k	l	m	n	b

			c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL* **|** *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies a null-terminated string indicating the names of the characters supported at a specified position, when ControlValue2 is set to M_CHAR_LIST. Character names must refer to specific letters (such as 'o'), digits (such as '0'), and punctuation marks (such as '%') in the font. You cannot indicate a space when listing characters for M_CHAR_LIST. The specified string can contain one or more character names. List them all without separators. For example, to read the twenty-second, the ninth, and the third uppercase letters of the alphabet, use the string "VIC". Specifying multiple characters is equivalent to calling this function multiple times, and listing one character each time. You can list character names in hexadecimal format beginning with "\\x". This is necessary if your data is in an ASCII format and you want Unicode characters beyond the Basic Latin range. For example, Basic Latin does not include the smiley face character; to specify it, use "\\x263A". You can also list a string of character names with mixed notation; in this case, you should also use "\\x" (for example, "VIC\\x263A"). INQ														

The following ControlType and corresponding ControlValue1, ControlValue2, and ControlValuePtr parameter settings are used to generally control the different positions in the string model(s). In this case, set the StringModelLabelOrIndex parameter to the label or index of a string model (one or all).

Depending on the control type selected, you can control the default constraints for the positions in the string model (set the Position parameter to M_DEFAULT), or you can control the explicit constraints for a position (set the Position parameter to one or all positions). Note that positions have zero-based indexing.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For generally controlling the different positions in the string model			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlType		Description															
	ControlValue1																
	ControlValue2																
	ControlValuePtr																
M_CLONE_CONSTRAINTS_FROM		Clones the explicit constraints from the source to the destination. The string model position specified with the ControlValue1 and ControlValue2 parameters represents the source. The string model position specified with the StringModelLabelOrIndex and Position parameters represents the destination. If the destination is a specific position (one or all), it will be explicitly constrained according to the explicit constraints of the source. Explicit constraints already specified for the destination will be overwritten. Only the cloned ones will remain. If the destination refers to the default constraints for the positions in the string model, all its implicitly constrained positions will be identical to the explicitly constrained position identified by the source. The clone operation does not override the state of the positions in the source; they remain implicitly constrained.															
	ControlValue1	Specifies the string model that has the explicit constraints to clone.	c		e	f	g	a		i		j	k	l	m	n	b
		M_STRING_INDEX(AIL_INT IndexValue)	Specifies the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
			Parameters														
			IndexValue Specifies the index of the string model.														

| | | |

		0 <= *Value* <= 255 Specifies the position in the string model.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DELETE_PERMITTED_CHARS_ENTRY		Deletes a character constraint at a specific index.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValue1	Specifies which character constraint to delete.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to delete all character constraints. The permitted character constraints revert to those of the default constraint.	c		e	f	g	a		i		j	k	l	m	n	b
	0<=*Value*<M_NUMBER_OF_PERMITTED_CHARS_ENTRIES	Specifies to delete the character constraint at the specified index.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_IS_OPTIONAL	Sets whether a position in a string model is optional. You should mark a model position as optional if you do not require a character to be returned for that position in the model. If the string model has a minimum string size (M_STRING_SIZE_MIN) less than its maximum size (M_STRING_SIZE_MAX), optional positions can be skipped if their constraints cannot be met (up to a maximum of M_STRING_SIZE_MAX- M_STRING_SIZE_MIN skips). For example, if you want to read 10-digit telephone numbers in the formats '###-###-####' and '##########', you can set the minimum string size to 10, set the maximum string size to 12, and mark positions 3 and 7 as optional. Then, set the permitted character constraints to M_DIGITS for positions 0-2, 4-6, and 8-11 and to hyphens for positions 3 and 7. If the target string is "123-456-7890", all the constraints are met, and SureDotOCR will return "123-456-7890". If the target string is "1234567890", positions 3 and 7 are skipped because they are optional and their constraints cannot be met, and SureDotOCR will return "1234567890". Note that unless you set a model position as optional, its character constraints must be met to return the string. Regardless of whether a model position is optional, if the target image does not meet the constraints and M_SPACE_SIZE_MIN is set wide enough, SureDotOCR will ignore the character and check if the constraints are met by the next character. If the distance between the previous character and the next character is less than the minimum space width, the current character is ignored and the next character can be used to meet the constraints. In the example above, if you had set the model to '##########', the target string "123-456-7890" might still have been read (as "1234567890") if the minimum space width was wide enough. It is always safer to include optional characters in the model and mark their positions as optional. You can use MdmrGetResult() with M_SKIPPED_POSITIONS to see which positions in the model were skipped. Note that a string read with a lower number of skips will be returned as a higher priority result than a string read with a higher number of skips. Note that MdmrPreprocess() will not accept optional constraints if M_STRING_PARTIAL_MODE is enabled. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Specifies whether the model position is optional.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the model position is not optional.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the model position is optional.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET_IMPLICIT_CONSTRAINTS	Resets the implicit (default) constraints for the positions in the string model back to their initial value. You can only use M_RESET_IMPLICIT_CONSTRAINTS if the Position parameter is set to M_DEFAULT. Initially, any character from any font can be read at every string model position.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValue1	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b

	ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET_POSITION_TO_IMPLICIT_CONSTRAINTS		Resets the explicitly constrained position(s) to the implicit (default) constraints for the string model. These positions are then considered implicitly constrained. You can only use M_RESET_POSITION_TO_IMPLICIT_CONSTRAINTS if the Position parameter is not set to M_DEFAULT. The constrained order (M_POSITION_CONSTRAINED_ORDER()) of a position changes when you use M_RESET_POSITION_TO_IMPLICIT_CONSTRAINTS.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValue1	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValue2	Set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MdmrControlStringModelInt64 (AIL_ID ContextDmrId, AIL_INT64 StringModelLabelOrIndex, AIL_INT64 Position, AIL_INT64 ControlType, AIL_INT64 ControlValue1, AIL_INT64 ControlValue2, const void *ControlValuePtr)
```

Details

Parameters

ContextDmrId

See ContextDmrId of the main function for a description.

StringModelLabelOrIndex

See StringModelLabelOrIndex of the main function for a description.

Position

See Position of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue1

See ControlValue1 of the main function for a description.

ControlValue2

See ControlValue2 of the main function for a description.

ControlValuePtr

See ControlValuePtr of the main function for a description.

```
void MdmrControlStringModelDouble (AIL_ID ContextDmrId, AIL_INT64 StringModelLabelOrIndex, AIL_INT64 Position, AIL_INT64 ControlType,
AIL_DOUBLE ControlValue1, AIL_DOUBLE ControlValue2, const void *ControlValuePtr)
```

Details

Parameters

ContextDmrId

See ContextDmrId of the main function for a description.

StringModelLabelOrIndex

See StringModelLabelOrIndex of the main function for a description.

Position

See Position of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue1

See ControlValue1 of the main function for a description.

ControlValue2

See ControlValue2 of the main function for a description.

ControlValuePtr

See ControlValuePtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrDraw

Synopsis

Draw specific features of a SureDotOCR result buffer.

Syntax

```
void MdmrDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ResultDmrId,            //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT64 Index,               //in
    AIL_INT64 CharIndex,           //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of a SureDotOCR result buffer in the destination image buffer or 2D graphics list.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultDmrId

Specifies the identifier of the SureDotOCR result buffer from which to extract the features to draw. The result buffer must have been previously allocated on the system using MdmrAllocResult().

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Operation

Specifies the drawing operation to perform. When applicable, **MdmrDraw**() draws the required information at the location that the string was read in the target, at the correct angle, scale, and aspect ratio.

For a drawing operation related to strings, you can set the Operation parameter to one of the following. The CharIndex parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a drawing operation related to strings																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DRAW_AIL_FONT_FORMATTED_STRING	Draws the names of the characters that were read, and they appear according to how the target string is formatted in the image (including non-constrained spaces). Character names are drawn under the bottom-left corner of the bounding box of the string (M_DRAW_STRING_BOX). Character names come from the SureDotOCR font that defines them. To draw the names, SureDotOCR uses the font associated with the 2D graphics context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_AIL_FONT_STRING	Draws the names of the characters that were read, and they appear according to the string model definition. Character names are drawn under the bottom-left corner of the bounding box of the string (M_DRAW_STRING_BOX). Character names come from the SureDotOCR font that defines them. To draw the names, SureDotOCR uses the font associated with the 2D graphics context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_BOX	Draws a bounding box around the string that was read. This bounding box encloses all of the string's characters.	c		e	f	g	a		i		j	k	l	m	n	b

For a drawing operation related to strings, you can set the Operation parameter to one of the following. The CharIndex parameter must be set to one or all characters.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a drawing operation related to characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_STRING_CHAR_BOX	Draws the character's bounding box at the location at which the character was read.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CHAR_POSITION	Draws a circled dot at the center of each bounding box that surrounds the individual characters of a string.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies the string result (one or all) for which to perform the drawing operation. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to perform the drawing operation for all string results.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to perform a global drawing operation for a single bounding box that encompasses all strings. This parameter must be used with M_DRAW_STRING_BOX or it will generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string result for which to perform the drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b

CharIndex

Specifies the character (one or all) in the string result for which to perform the drawing operation. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Implements the default behavior. For drawing operations related to strings, the default indicates that this parameter is not required. For drawing operations related to characters, the default is the same as setting the M_INDEX_IN_STRING() macro to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b

M_INDEX_IN_FORMATTED_STRING(AIL_INT CharIndex)	Specifies to perform the drawing operation by indicating the character's index, relative to its position in the formatted string. The formatted string includes spaces, if any were read in the target string, even if you did not add spaces to the string model as permitted character constraints (MdmrControlStringModel() with M_SPACE).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CharIndex Set this parameter to one of the following:															
	M_ALL — Specifies to perform the drawing operation for all characters in the formatted string. To retrieve the number of characters in the formatted string, call MdmrGetResult() with M_FORMATTED_STRING_CHAR_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_FORMATTED_STRING_CHAR_NUMBER — Specifies the index of the character in the formatted string for which to perform the drawing operation. The value must be less than the total number of characters in the formatted string (M_FORMATTED_STRING_CHAR_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_IN_STRING(AIL_INT CharIndex)	Specifies to perform the drawing operation by indicating the character's index, relative to its position in the string. The string can include spaces, but only if they correspond to a space permitted character constraint in the string model (MdmrControlStringModel() with M_SPACE).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CharIndex Set this parameter to one of the following:															
	M_ALL — Specifies to perform the drawing operation for all characters in the string. To retrieve the number of characters in the string, call MdmrGetResult() with M_STRING_CHAR_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_STRING_CHAR_NUMBER — Specifies the index of the character in the string for which to perform the drawing operation. The value must be less than the total number of characters in the string (M_STRING_CHAR_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrExportFont

Synopsis

Export a font from a SureDotOCR context.

Syntax

```
void MdmrExportFont(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 FileFormat,        //in
    AIL_ID ContextDmrId,         //in
    AIL_INT64 FontLabelOrIndex,  //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function exports a font from a SureDotOCR context to a file. The exported font includes all its characters. String models use fonts to read dot-matrix text. Once you export a font, you can import it to any SureDotOCR context using MdmrImportFont().

Parameters

FileName

Specifies the name and path of the SureDotOCR font file in which to export the font. The function handles (internally) the opening and closing of the file. For easier use with other Aurora Imaging Library software products, the specified file name must use the MDMRF file extension, which refers to a SureDotOCR font file. If you specify a file that already exists, it will be overwritten.

Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which to interactively specify the drive, directory, and name of a SureDotOCR font file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of a SureDotOCR font file (for example, "*C:\mydirectory\MySweetSweetFont.mdmrf*"). To specify a SureDotOCR font file on a remote computer (under Distributed Aurora Imaging Library), prefix the file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MySweetSweetFont.mdmrf*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat

Specifies the format of the file in which to export the font. Set this parameter to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DMR_FONT_FILE	Specifies a SureDotOCR font file format (MDMRF). See MdmrImportFont() for information on how SureDotOCR font files represent their data.	c		e	f	g	a		i		j	k	l	m	n	b

ContextDmrId

Specifies the identifier of the SureDotOCR context from which to export the font. The context must have been previously allocated on the system using MdmrAlloc().

FontLabelOrIndex

Specifies the label or index of the font to export from the SureDotOCR context. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For identifying the font																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_INDEX(AIL_INT IndexValue)	Specifies to export the font by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_FONTS — Specifies the index of the font to export.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL(AIL_INT LabelValue)	Specifies to export the font by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Set this parameter to the following:															
	0 < *Value* < 2097152 — Specifies the label of the font to export.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrFree

Synopsis

Free a SureDotOCR context or a SureDotOCR result buffer.

Syntax

```
void MdmrFree(
    AIL_ID ContextOrResultDmrId //in
)
```

Description

This function deletes the specified SureDotOCR context or SureDotOCR result buffer identifier, and releases any memory associated with it.

All SureDotOCR contexts and all SureDotOCR result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResultDmrId

Specifies the identifier of the SureDotOCR context or SureDotOCR result buffer to free. These must have been successfully allocated (with MdmrAlloc() or MdmrAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrGetResult

Synopsis

Get results from a SureDotOCR result buffer.

Syntax

```
void MdmrGetResult(
    AIL_ID ResultDmrId,          //in
    AIL_INT64 StringIndex,       //in
    AIL_INT64 CharPositionIndex, //in
    AIL_INT64 ResultType,        //in
    void *ResultArrayPtr         //out
)
```

Description

This function retrieves results of the specified type from a SureDotOCR result buffer. You can retrieve global results, string results, or character results. Results are available after calling MdmrRead().

If your target image (MdmrRead()) was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the center of top-left pixel in the target image.

If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MdmrControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MdmrGetResult**() generates an error.

Positional and dimensional results take into account the angle at which the result was found. The following example shows how the result for the bottom-left corner of the string box (M_STRING_BOX_BL_X and M_STRING_BOX_BL_Y) depends on the location and angle at which the string was read.

To remove all results from a result buffer, call MdmrControl() with M_RESET. This does not delete the buffer identifier, as is the case with MdmrFree().

Parameters

ResultDmrId

Specifies the identifier of the SureDotOCR result buffer from which to retrieve results.

StringIndex

Specifies the string (one or all) for which to get results, or specifies to get global results. Set this parameter to one of the values below.

Note that you must set this parameter to a string (one or all) when getting character results.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string for which to get results, or for specifying global results																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to retrieve results for all strings in the result buffer. To retrieve the number of strings in the result buffer, use M_STRING_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve global results. These results are related to the result buffer itself.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_STRING_NUMBER	Specifies the index of the string for which to retrieve results. The index must be less than the total number of strings in the result buffer (M_STRING_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b

CharPositionIndex

Specifies the character (one or all) for which to get results, or specifies to get global or string results. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character for which to get results, or for specifying global or string results

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_IN_FORMATTED_STRING(AIL_INT CharIndex)	Specifies to retrieve a character result by indicating the character's index, according to the target string format. The StringIndex parameter must be set to a string (one or all). If a space is read in the target string, it is indexed as a character result, even if you did not add a space to the string model as a permitted character constraint (MdmrControlStringModel() with M_SPACE). For example, if the string model definition specifies to read 4 capital letters ("LLLL"), and the target string read is "AB CD", M_INDEX_IN_FORMATTED_STRING(3) refers to 'C'. If the string model definition specifies to read 4 capital letters and 1 space ("LLSLL"), and the target string read is "AB CD", M_INDEX_IN_FORMATTED_STRING(3) also refers to 'C'.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CharIndex Set this parameter to one of the following:															
	M_ALL — Specifies to retrieve results for all characters. To retrieve the number of characters, use M_FORMATTED_STRING_CHAR_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_FORMATTED_STRING_CHAR_NUMBER — Specifies the index of the character for which to get results. The value must be less than the total number of characters (M_FORMATTED_STRING_CHAR_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_IN_STRING(AIL_INT CharIndex)	Specifies to retrieve a character result by indicating the character's index, according to the string model definition. The StringIndex parameter must be set to a string (one or all). Spaces read in the target string can be indexed as a character result, but only if they correspond to a space permitted character constraint in the string model (MdmrControlStringModel() with M_SPACE). For example, if the string model definition specifies to read 4 capital letters ("LLLL"), and the target string read is "AB CD", M_INDEX_IN_STRING(3) refers to 'D'. If the string model definition specifies to read 4 capital letters and 1 space ("LLSLL"), and the target string read is "AB CD", M_INDEX_IN_STRING(3) refers to 'C'.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CharIndex Set this parameter to one of the following:															
	M_ALL — Specifies to retrieve results for all characters. To retrieve the number of characters, use M_STRING_CHAR_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_STRING_CHAR_NUMBER — Specifies the index of the character for which to get results. The value must be less than the total number of characters (M_STRING_CHAR_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve global results or string results. M_GENERAL or M_DEFAULT retrieves global results when the StringIndex parameter is set to M_GENERAL or M_DEFAULT. M_GENERAL or M_DEFAULT retrieves string results when the StringIndex parameter is set to a string (one or all).	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving global results
- For retrieving string results
- For retrieving global or string results
- For retrieving character results

When retrieving global results, set the ResultType parameter to the value below. In this case, set both the StringIndex and CharPositionIndex parameters to either M_GENERAL or M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving global results																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_STATUS +		Retrieves information regarding the state of the read operation.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPLETE	Specifies that the read operation completed successfully. It is possible to read 0 strings (M_STRING_NUMBER), and have a successfully completed read operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CURRENTLY_READING	Specifies that the read operation is currently ongoing. You can only get this status if you are retrieving it from another thread.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERNAL_ERROR	Specifies that an unexpected error occurred.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_ENOUGH_MEMORY	Specifies that a memory allocation error occurred while reading.	c		e	f	g	a		i		j	k	l	m	n	b
	M_READ_NOT_PERFORMED	Specifies that the read operation has not been performed. This is the initial status.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STOPPED_BY_REQUEST	Specifies that the read operation was explicitly stopped (MdmrControl() with M_STOP_READ).	c		e	f	g	a		i		j	k	l	m	n	b

M_TIMEOUT_REACHED	Specifies that the read operation ended because the timeout limit was reached (MdmrControl() with M_TIMEOUT).	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_NUMBER +	Retrieves the total number of strings read. This is the total number of strings held by the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving string results, set the ResultType parameter to one of the values below. In this case, set the StringIndex parameter to the index of the string for which to get the result or M_ALL (unless otherwise specified), and set the CharPositionIndex parameter to either M_GENERAL or M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving string results																	
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FORMATTED_STRING +	Retrieves the string that was read, according to the target string format. This information is returned as a regular string. It includes the name of all the characters in the string and the terminating null character. It also includes spaces, if the target string has any that are greater than the minimum spacing (MdmrControl() with M_SPACE_SIZE_MIN) and less than the maximum spacing (M_SPACE_SIZE_MAX). Note that this result type cannot be used when the StringIndex parameter is set to M_ALL.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b
M_FORMATTED_STRING_CHAR_INVALID_INDICES +	Retrieves the positions of invalid or unreadable characters in the formatted string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FORMATTED_STRING_CHAR_NUMBER +	Retrieves the total number of characters read, according to the target string format. This number includes spaces, if any were read in the target string, even if you did not add spaces to the string model as permitted character constraints (MdmrControlStringModel() with M_SPACE). M_FORMATTED_STRING_CHAR_NUMBER does not count the terminating null character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SKIPPED_POSITIONS +	Retrieves the positions of the skipped characters; this result is only available when M_STRING_SIZE_MIN is less than M_STRING_SIZE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING +	Retrieves the string that was read, according to the string model definition. This information is returned as a regular string. It includes the name of all the characters in the string and the terminating null character. It also include spaces, if they correspond to a space permitted character constraint in the string model (MdmrControlStringModel() with M_SPACE). Note that this result type cannot be used when the StringIndex parameter is set to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_CHAR_INVALID_INDICES +	Retrieves the positions of invalid or unreadable characters in the string. To retrieve the positions of these characters in the formatted string, use M_FORMATTED_STRING_CHAR_INVALID_INDICES.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_NUMBER +	Retrieves the total number of characters read, according to the string model definition. This number can include spaces, but only if they correspond to a space permitted character constraint in the string model (MdmrControlStringModel() with M_SPACE). M_STRING_CHAR_NUMBER does not count the terminating null character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_MODEL_INDEX +	Retrieves the index of the string model, in the SureDotOCR context, used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string model used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_MODEL_LABEL +	Retrieves the label of the string model, in the SureDotOCR context, used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string model used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCORE +	Retrieves the string score calculated during the read operation. The string score is the average score of the characters in the string. This score does not include the score of an M_SPACE permitted character (MdmrControlStringModel()), if any were read in the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving global or string results (one or all), set the ResultType parameter to one of the values below. In this case, set the StringIndex parameter to one or all strings, M_GENERAL or M_DEFAULT, and set the CharPositionIndex parameter to either M_GENERAL or M_DEFAULT.

> Note when the StringIndex parameter is set to either M_GENERAL or M_DEFAULT, the result type will be in reference to the region that encompasses all strings, as opposed to the region of the specific string(s).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving global or string results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_ITALIC_CHAR_ANGLE +	Retrieves the italic angle at which characters in the string(s) are found.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_PITCH +	Retrieves the distance between successive dot centers in the italic angle direction.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE +	Retrieves the angle of the string's bounding box, or the bounding box encompassing all strings, in degrees. The angle is measured at the box's base and is relative to the output coordinate system (MdmrControl() with M_RESULT_OUTPUT_UNITS). An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise, from the positive X-axis towards the negative Y-axis of the image. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BL_X +	Retrieves the X-coordinate of the bottom-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BL_Y +	Retrieves the Y-coordinate of the bottom-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BR_X +	Retrieves the X-coordinate of the bottom-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BR_Y +	Retrieves the Y-coordinate of the bottom-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UL_X +	Retrieves the X-coordinate of the top-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UL_Y +	Retrieves the Y-coordinate of the top-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UR_X +	Retrieves the X-coordinate of the top-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UR_Y +	Retrieves the Y-coordinate of the top-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_ANGLE +	Retrieves the angle at which characters in the string(s) are found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_HEIGHT +	Retrieves the height of the string's bounding box, or the bounding box encompassing all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PITCH +	Retrieves the distance between successive dot centers in the string angle direction.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_POSITION_X +	Retrieves the X-coordinate of the center of the bounding box of the string, or the bounding box encompassing all strings; this position is considered the position of the string, or bounding box of all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_POSITION_Y +	Retrieves the Y-coordinate of the center of the bounding box of the string, or the bounding box encompassing all strings; this position is considered the position of the string, or bounding box of all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_WIDTH +	Retrieves the width of the string's bounding box, or the bounding box encompassing all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving results for characters in a string, set the ResultType parameter to one of the values below. In this case, set the StringIndex parameter to a string (one or all), and set the CharPositionIndex parameter to a character (one or all). Characters are ordered by their index position in their string.

> Note that M_CHAR_NAME cannot be used when the StringIndex parameter is set to M_ALL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving character results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_CHAR_BOX_BL_X +	Retrieves the X-coordinate of the bottom-left corner of the character's bounding box, relative to the angle at which the character was found.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BL_Y +	Retrieves the Y-coordinate of the bottom-left corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BR_X +	Retrieves the X-coordinate of the bottom-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BR_Y +	Retrieves the Y-coordinate of the bottom-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_BOX_UL_X +	Retrieves the X-coordinate of the top-left corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UL_Y +	Retrieves the Y-coordinate of the top-left corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UR_X +	Retrieves the X-coordinate of the top-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UR_Y +	Retrieves the Y-coordinate of the top-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_FONT_INDEX +	Retrieves the index of the font, in the SureDotOCR context, that contains the representation of the character that was read in the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_FONT_LABEL +	Retrieves the label of the font, in the SureDotOCR context, that contains the representation of the character that was read in the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HEIGHT +	Retrieves the height of the character's bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NAME +	Retrieves the name of the character in the string that was read. The name is returned as a regular string. It consists of the character name and the terminating null character. If a space was read, the name returned is an actual space (and the terminating null character).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_X +	Retrieves the X-coordinate of the center of the bounding box of the character; this position is considered the position of the character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_Y +	Retrieves the Y-coordinate of the center of the bounding box of the character; this position is considered the position of the character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCORE +	Retrieves the character's score. This quantifies the similarity between the character in the target and the character in the font. If SureDotOCR reads an M_SPACE permitted character (MdmrControlStringModel()), its character score is always 100%. This score does not influence the string score (M_STRING_SCORE).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_WIDTH +	Retrieves the width of the character's bounding box.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_FORMATTED_STRING; M_STRING; M_CHAR_NAME.

You can add the following value to the above-mentioned values to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the size of a string																		
ResultType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_STRING_SIZE +		Retrieves the number of characters in the string, including the terminating null character. When specifying M_FORMATTED_STRING + M_STRING_SIZE, the number returned also include spaces, if any were read in the target string. When specifying M_STRING + M_STRING_SIZE, the number returned can also include spaces, but only if they correspond to a space permitted character constraint in the string model. When specifying M_CHAR_NAME + M_STRING_SIZE, the number returned can include a space, if that was the character read. For M_CHAR_NAME, the name of each letter at each position is itself a string, with its own terminating null character. You must sum them all to get the required array size for a string.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_FORMATTED_STRING; M_STRING; M_CHAR_NAME; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to determine the hexadecimal 16-bit Unicode (UTF-16) character name information.

The character name of the letter A is "A" (its native name) or "\x0041", which is its representation in hexadecimal format beginning with "\x". Characters between 0 and 127 are considered Basic Latin. A character such as the smiley face (☺, or "\x263A" in hexadecimal) is beyond the Basic Latin range.

In a Unicode environment, the character representations are returned (for example, the smiley face), unless you explicitly specify one of the below combination values (for example, to obtain "\x263A" instead of the actual smiley face). Retrieving character names in hexadecimal might be useful if you want to convert your data into ASCII.

Note how the string size is affected. For example, the string "abc☺" nominally consists of four characters plus the null-terminating character. Accordingly, a string size result (retrieved with, for example, M_STRING + M_STRING_SIZE) will return 5 as the string size. For the same string mentioned above ("abc☺"), when M_HEX_UTF16_FOR_ALL is specified, you will obtain "\x0041\x0042\x0043\x263A" and a string size of 25. When M_HEX_UTF16_FOR_NON_BASIC_LATIN is specified, you will obtain "abc\x263A" and a string size of 10.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For explicitly retrieving character name information in hexadecimal 16-bit Unicode (UTF-16) format																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HEX_UTF16_FOR_ALL	Retrieves results with the name of all characters in hexadecimal 16-bit Unicode (UTF-16) format.	c		e	f	g	a		i		j	k	l	m	n	b
M_HEX_UTF16_FOR_NON_BASIC_LATIN	Retrieves results with the names of characters that fall out of the Basic Latin range, in hexadecimal 16-bit Unicode (UTF-16) format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving global results
- For retrieving string results
- For retrieving global or string results
- For retrieving character results

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For retrieving string results, For retrieving global or string results, For retrieving character results; (exceptions might exist) and for the following values: M_FORMATTED_STRING_CHAR_INVALID_INDICES; M_STRING_CHAR_INVALID_INDICES.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For retrieving global results, For retrieving global or string results; and for the following values: M_FORMATTED_STRING_CHAR_INVALID_INDICES; M_FORMATTED_STRING_CHAR_NUMBER; M_SKIPPED_POSITIONS; M_STRING_CHAR_INVALID_INDICES; M_STRING_CHAR_NUMBER; M_STRING_MODEL_INDEX; M_STRING_MODEL_LABEL; M_STRING_SCORE; M_CHAR_BOX_BL_X; M_CHAR_BOX_BL_Y; M_CHAR_BOX_BR_X; M_CHAR_BOX_BR_Y; M_CHAR_BOX_UL_X; M_CHAR_BOX_UL_Y; M_CHAR_BOX_UR_X; M_CHAR_BOX_UR_Y; M_CHAR_FONT_INDEX; M_CHAR_FONT_LABEL; M_CHAR_HEIGHT; M_CHAR_POSITION_X; M_CHAR_POSITION_Y; M_CHAR_SCORE; M_CHAR_WIDTH; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the result to a required data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrImportFont

Synopsis

Import characters from a font file into a new or existing font of a SureDotOCR context.

Syntax

```
void MdmrImportFont(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 FileFormat,        //in
    AIL_ID ContextDmrId,         //out
    AIL_INT64 FontLabelOrIndex,  //in
    AIL_CONST_TEXT_PTR CharList, //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function imports characters from a font file into a new or existing font of a SureDotOCR context. The fonts of a context contain the characters that string models use to read dot-matrix text. Fonts should only contain characters you want to read.

For easier use with other Aurora Imaging Library software products, you should import characters from a SureDotOCR font file. This is a text file saved with the extension MDMRF. The file must be in UTF-16 (little-endian order) with a BOM (Byte Order Mark) indicating this; the BOM value is U+FEFF. You can also import a font from a SureDotOCR font file exported from a SureDotOCR context, using MdmrExportFont().

The content of SureDotOCR font files must adhere to the expected format. Below is an example of how the character 'A' should appear in a SureDotOCR font file.

```
File Representation
AIL_DOT_FONT

CharValue A
00 00 FF 00 00
00 FF 00 FF 00
FF 00 00 00 FF
FF 00 00 00 FF
FF FF FF FF FF
FF 00 00 00 FF
FF 00 00 00 FF
```

The BOM U+FEFF is written as invisible text according to the settings of the tool used to save the MDMRF file. Since the file must use little-endian encoding, the BOM U+FEFF is represented in the file's first two bytes as 0xFF 0xFE.

SureDotOCR font files should have "File Representation" and "AIL_DOT_FONT" in lines 1 and 2. Lines 4 and greater should specify the characters; separate each with a blank line. Every character should begin with the word "CharValue" and its name. Names can be in UTF-16 byte hexadecimal by beginning them with "\x" (for example, 'A' would use "\x0041"). The character's dot-matrix must be below the "CharValue" line. Use "FF" for dots that represent the character; otherwise use "00". The dot-matrix of the font's characters should be identical to the dot-matrix of the characters in the strings to read. Every dot-matrix in a font must have the same number of columns and rows.

Aurora Imaging Library installs SureDotOCR console-based font utilities, such as *DmrEditFontFile* (accessible from the Aurora Imaging Library Example Launcher). Predefined SureDotOCR font files (such as **.mdmrf* files) are typically distributed with Aurora Imaging Library and can be found in the installed Contexts folder (for example, *C:\Program Files\Aurora Imaging Library\11\Contexts*"). Use them to create and update fonts. In this case, file and content requirements are automatically respected; ensure that you maintain these requirements as you develop your fonts. For more information, see the Fonts section of Chapter 15: SureDotOCR.

When you call **MdmrImportFont**() to add a new font (or to add characters to an empty font), the dimensions of the characters' dot-matrix represent the dimensions of the dot-matrix template of the font. Subsequent characters that you add to the font must use these dimensions or you will get an error. To inquire the dimensions, call MdmrInquireFont() with M_FONT_SIZE_COLUMNS and M_FONT_SIZE_ROWS. You can also use this function to inquire about other aspects of the font.

To delete a font from a context, or to delete a character from a font, call MdmrControl() or MdmrControlFont(), respectively. You can also use these functions to set font-related control types, add an empty font to a context, or add characters to a font (or modify existing characters) by defining their representations in an array.

> Fonts should only contain characters you want to read. Delete all unnecessary characters.

A SureDotOCR font file must be for a single font. To import multiple fonts, call **MdmrImportFont**() multiple times. Modifying a context's fonts can affect which strings are read in the target image. You must preprocess the SureDotOCR context after you have finished adjusting a context's fonts and before calling MdmrRead(). To know if a context needs to be preprocessed, call MdmrInquire() with M_PREPROCESSED.

You cannot represent certain characters, such as null or the standard space ("0x0020"), in a font file. To establish the distance between characters in a target string that represents a space, call MdmrControl() with M_SPACE_SIZE_MAX and M_SPACE_SIZE_MIN.

Parameters

FileName

Specifies the name and path of the SureDotOCR font file (MDMRF) from which to import the font. The function internally handles the opening and closing of the file. To specify the file name and path of a SureDotOCR font file, set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of a SureDotOCR font file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of a SureDotOCR font file (for example, "*C:\mydirectory\MySweetSweetFont.mdmrf*"). To specify a SureDotOCR font file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MySweetSweetFont.mdmrf*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat

Specifies the format of the file from which to import the font. Set this parameter to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DMR_FONT_FILE	Specifies a SureDotOCR font file format (MDMRF).	c		e	f	g	a		i		j	k	l	m	n	b

ContextDmrId

Specifies the identifier of the SureDotOCR context to which you are importing the characters. The context must have been previously allocated on the system using MdmrAlloc().

FontLabelOrIndex

Specifies the label or index of an already existing font, or the addition of a new font. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying an already existing font, or the addition of a new font																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_NEW_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_INDEX(AIL_INT IndexValue)	Specifies an already existing font in the context by indicating its index. SureDotOCR imports the new characters to the specified font.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_FONTS — Specifies the index of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL(AIL_INT LabelValue)	Specifies to add a new font or to update an already existing font by indicating a label. If the specified label refers to a font in the context, SureDotOCR imports the new characters to it. If the specified label does not refer to a font in the context, SureDotOCR adds a new font to the context with that label, and imports the new characters to it.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Set this parameter to the following:															

	Value > 0 Specifies the label of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEW_LABEL	Specifies to add the imported characters as a new font in the context and to automatically assign it a label. To inquire about the label, use MdmrInquireFont() with M_FONT_LABEL_VALUE. To change the label, use MdmrControlFont() with M_FONT_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b

CharList

Specifies the characters to import from the SureDotOCR font file. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the characters to import																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Same as M_IMPORT_ALL_CHARS.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMPORT_ALL_CHARS	Imports all characters.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*CharList*") 1	Specifies a null-terminated string indicating the names of the characters to import. Character names must refer to specific letters (such as 'o'), digits (such as '0'), and punctuation marks (such as '%') in the font. You cannot indicate a space. The specified string can contain one or more character names. List them all without separators. For example, to specify the twenty-second, the ninth, and the third uppercase letters of the alphabet, use the string "VIC". Specifying multiple characters is equivalent to calling this function multiple times, and listing one character each time. You can list character names in hexadecimal format beginning with "\x". This is necessary if your data is in an ASCII format and you want Unicode characters beyond the Basic Latin range. For example, Basic Latin does not include the smiley face character; to specify it, use "\x263A". You can also list a string of character names with mixed notation; in this case, use "\\x" instead (for example, "VIC\\x263A").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ControlFlag

Specifies how to handle importing characters that have the same name as characters already in the font. A font in a context cannot have multiple characters with the same name. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to handle characters with the same name																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_NO_OVERWRITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_OVERWRITE	Specifies to not overwrite characters already in the font with characters that you are importing, when the characters have the same name.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERWRITE	Specifies to overwrite characters already in the font with characters that you are importing, when the characters have the same name.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrInquire

Synopsis

Inquire about a SureDotOCR context or result buffer setting.

Syntax

```
AIL_INT MdmrInquire(
    AIL_ID ContextOrResultDmrId, //in
    AIL_INT64 InquireType,       //in
    void *UserVarPtr             //out
)
```

Description

This function inquires about a setting of a SureDotOCR context or result buffer. Such settings are typically specified with MdmrControl().

If the inquired setting is set to M_DEFAULT (for example, in MdmrControl()), **MdmrInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextOrResultDmrId

Specifies the identifier of the SureDotOCR context or result buffer about which to inquire. The context must have been previously allocated on the system using MdmrAlloc(). The result buffer must have been previously allocated on the system using MdmrAllocResult().

InquireType

Specifies the setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MdmrInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a SureDotOCR context
- For inquiring about a SureDotOCR result buffer

To inquire about the SureDotOCR context specified with the ContextOrResultDmrId parameter, set the InquireType parameter to one of the values below. SureDotOCR can only read strings that adhere to these settings (they apply to all string models in the context).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a SureDotOCR context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_DOT_DIAMETER +		Inquires the diameter of the dots that make up the characters in the strings. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies an invalid diameter. Set the diameter to a valid value, using MdmrControl() with M_DOT_DIAMETER.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the diameter, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_DIAMETER_SPREAD +		Inquires the range of possible dot diameter deviations, in pixels, centered at the M_DOT_DIAMETER. In other words, the maximum allowed dot diameter deviation over and below the specified dot diameter is half this value.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the spread, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_DIAMETER_SPREAD_MODE +		Inquires whether the range of possible dot diameter deviations is enabled, or not.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that range of dot diameter deviations is disabled.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that range of dot diameter deviations is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOT_DIAMETER_STEP +		Inquires the step that will be used inside the range of dot diameter deviation.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the step, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +		Inquires the foreground value of the characters in the strings. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FOREGROUND_BLACK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_BLACK	Specifies that strings with characters darker than the background will be read. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_WHITE	Specifies that strings with characters lighter than the background will be read. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_ANGLE +		Inquires the italic angle at which to read the characters in the strings. M_ITALIC_ANGLE only has an effect if M_ITALIC_ANGLE_MODE is set to M_ANGLE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

-90.0 <= *Value* <= 90.0	Specifies the italic angle, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_ANGLE_MODE +	Inquires whether the italic angle is detected automatically or explicitly specified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE	Specifies to use the italic angle specified with M_ITALIC_ANGLE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the italic angle is detected automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_PITCH +	Inquires the distance between successive dot centers, in the italic angle direction, at which to read the characters in the strings. M_ITALIC_PITCH only has an effect if M_ITALIC_PITCH_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the pitch, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ITALIC_PITCH_MODE +	Inquires whether the distance between successive dot centers, in the italic angle direction, is detected automatically or explicitly specified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the distance between dot centers in the italic angle direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the value set with M_ITALIC_PITCH. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY +	Inquires the maximum pixel intensity of a character's dots. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 255.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 255.0	Specifies the maximum pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY_MODE +	Inquires how to establish the maximum pixel intensity of a character's dots. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the maximum pixel intensity of a character's dots. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the maximum pixel intensity value set with M_MAX_INTENSITY.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_CONTRAST +	Inquires the minimum contrast between a character and its background. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 15.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 255.0	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_CONTRAST_MODE +	Inquires how to establish the minimum contrast between a character and its background. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the minimum contrast of characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the minimum contrast value set with M_MIN_CONTRAST.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_INTENSITY +	Inquires the minimum pixel intensity of a character's dots. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 255.0	Specifies the minimum pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_INTENSITY_MODE +	Inquires how to establish the minimum pixel intensity of a character's dots. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the minimum pixel intensity of character dots. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the minimum pixel intensity value set with M_MIN_INTENSITY.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_COUNT +	Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context are modified. Although you cannot identify the modification counter's contents, you can compare them throughout your application to know if the context has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FONTS +	Inquires the number of fonts in the context. To add and remove fonts, use MdmrControl(). To import fonts, use MdmrImportFont().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of fonts.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_STRING_MODELS +	Inquires the number of string models in the context. To add and remove string models, use MdmrControl().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of string models.	c		e	f	g	a		i		j	k	l	m	n	b

M_OWNER_SYSTEM +	Inquires the identifier of the system on which the context was allocated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
Aurora Imaging Library system identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +	Inquires whether the context is preprocessed. The context must be preprocessed, using MdmrPreprocess(), before calling MdmrRead(). After certain modifications to the context, as can be done with MdmrControl(), MdmrControlFont(), or MdmrControlStringModel(), this inquire type will indicate that the context is no longer in its preprocessed state.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the context is not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the context is preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MAX +	Inquires the value with which to determine the maximum width of a space character. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum width value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MAX_MODE +	Inquires how to determine the maximum width of a space character. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_CHAR_WIDTH_FACTOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_WIDTH_FACTOR	Specifies that the maximum width of a space character is determined by multiplying the M_SPACE_SIZE_MAX value by the string's maximum character width, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies no maximum space width. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MIN +	Inquires the value with which to determine the minimum width of a space character. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the minimum width value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE_SIZE_MIN_MODE +	Inquires how to determine the minimum width of a space character. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_CHAR_WIDTH_FACTOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_WIDTH_FACTOR	Specifies that the minimum width of a space character is determined by multiplying the M_SPACE_SIZE_MIN value by the string's maximum character width, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies no minimum space width. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE +	Inquires the angle at which to read a string. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies that the angle is set to the angle of the target image's ROI. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies a fixed angle at which to read a string, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_INPUT_UNITS +	Inquires the coordinate system that is used for M_STRING_ANGLE. This is ignored when M_STRING_ANGLE is set to M_ACCORDING_TO_REGION. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, with respect to the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ANGLE_MODE +	Inquires whether the string angle is detected automatically or explicitly specified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE	Specifies to use the angle specified with M_STRING_ANGLE; the string will be read from left to right with the characters facing upward along the specified angle. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the string angle is detected automatically; M_STRING_ANGLE is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIENTATION	Specifies that the string will be read from left to right with the characters facing upward, or from right to left with the character facing downward. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PARTIAL_CHAR_INVALID +	Inquires the single character string to replace invalid or unrecognized characters if M_STRING_PARTIAL_MODE is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*SingleCharacterString*") 1	Specifies the single character string to replace invalid or unrecognized characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PARTIAL_MODE +	Inquires how partial strings are returned during the reading process.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that only complete strings will be returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the best match for a partial string will be returned; unrecognized characters in the string will be replaced with M_STRING_PARTIAL_CHAR_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_PITCH +	Inquires the distance between successive dot centers, in the string angle direction, at which to read the characters in the strings. M_STRING_PITCH only has an effect if M_STRING_PITCH_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 3.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the pitch, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_PITCH_MODE +	Inquires whether the distance between successive dot centers, in the string angle direction, is detected automatically or explicitly specified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically establish the distance between dot centers in the string angle direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to use the value set with M_STRING_PITCH. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_BLOCK_HEIGHT +	Inquires the height of the text block. This is the area that encloses all dot-matrix text in the image buffer (or in the buffer's associated region of interest). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid height. Set the height to a valid value, using MdmrControl() with M_TEXT_BLOCK_HEIGHT.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_BLOCK_SIZE_MODE +	Inquires how to establish the size of the text block. This is the area that encloses all dot-matrix text in the image buffer (or in the buffer's associated region of interest). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the size of the rectangular block is defined by the M_TEXT_BLOCK_HEIGHT and M_TEXT_BLOCK_WIDTH settings.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_BLOCK_WIDTH +	Inquires the width of the text block. This is the area that encloses all dot-matrix text in the image buffer (or in the buffer's associated region of interest). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an invalid width. Set the width to a valid value, using MdmrControl() with M_TEXT_BLOCK_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +	Inquires the maximum read time for MdmrRead(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2000.0 msec.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies an infinite amount of read time.	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0.0	Specifies the maximum read time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for M_STRING_PARTIAL_CHAR_INVALID.

You can add the following value to the above-mentioned value to get the size of a string..

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the size of a string																	
InquireType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE +		Retrieves the number of characters in the string, including the terminating null character.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the SureDotOCR result buffer specified with the ContextOrResultDmrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a SureDotOCR result buffer																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +		Inquires whether to return results in pixels or world units. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For inquiring about a SureDotOCR context, For inquiring about a SureDotOCR result buffer.

You can add the following value to the above-mentioned values to get the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEFAULT		Inquires the default value of the specified inquire type.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a SureDotOCR context, For inquiring about a SureDotOCR result buffer.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported for the SureDotOCR context currently being inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring if an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_HAS_DEFAULT	Inquires whether the specified inquire type has a default value.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED	Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a SureDotOCR context, For inquiring about a SureDotOCR result buffer; (exceptions might exist) and for the following value: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the result to a required data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested results to an *AIL_INT64.*	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrInquireFont

Synopsis

Inquire about a global setting of a font in a SureDotOCR context, or inquire about a character in a font.

Syntax

```
AIL_INT MdmrInquireFont(
    AIL_ID ContextDmrId,            //in
    AIL_INT64 FontLabelOrIndex,     //in
    AIL_INT64 CharIndex,            //in
    AIL_CONST_TEXT_PTR CharNamePtr, //in
    AIL_INT64 InquireType,          //in
    void *UserVarPtr                //out
)
```

Description

This function inquires about a global setting of a font in a SureDotOCR context, or inquires about a character in a font. Font settings can be specified with MdmrControlFont().

Characters you are inquiring about are typically imported into a font with MdmrImportFont(). You can also inquire about characters added to a font with MdmrControlFont().

If the inquired setting is set to M_DEFAULT (for example, in MdmrControlFont()), **MdmrInquireFont**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context that contains the font about which to inquire. The context must have been previously allocated on the system using MdmrAlloc().

FontLabelOrIndex

Specifies the font about which to inquire. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the font																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_INDEX(AIL_INT IndexValue)	Specifies to inquire about the font by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the font.															
	0 <= *Value* < M_NUMBER_OF_FONTS Specifies the index of the font to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL(AIL_INT LabelValue)	Specifies to inquire about the font by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Specifies the label of the font.															
	0 < *Value* < 2097152 Specifies the label of the font about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

CharIndex

Specifies the index of a character in the font, if required. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m –

rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of a character																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that this parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of a character.	c		e	f	g	a		i		j	k	l	m	n	b

CharNamePtr

Specifies the name of a character in the font, if required. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the character																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that this parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*CharName*") 1	Specifies the character name. For example, 'A' is the name that identifies the first uppercase letter of the English alphabet. When using the CharNamePtr parameter to specify a character, the CharIndex parameter must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

InquireType

Specifies the type of inquire to perform.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_UINT8

Specifies the address in which to write the requested information. Since the **MdmrInquireFont**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a global setting of a font
- For inquiring about a font's characters

To inquire about a global setting of a font, set the InquireType parameter to one of the values below. Unless otherwise specified, set the FontLabelOrIndex parameter to the label or index of a font, the CharIndex parameter to M_DEFAULT, and the CharNamePtr parameter to M_NULL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a global setting of a font																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_FONT_INDEX_VALUE +	Inquires the index value of a font. Set the FontLabelOrIndex parameter to the font's label.															

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the font to which you are referring does not exist in the context.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the index of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL_VALUE +	Inquires the label value of the font. Set the FontLabelOrIndex parameter to the font's index.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0 < *Value* < 2097152	Specifies the font's label value. If SureDotOCR cannot return the requested information (no such font exists in the context), you will get an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_SIZE_COLUMNS +	Inquires the number of columns in the font's dot-matrix template. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 5 columns.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of columns, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_SIZE_ROWS +	Inquires the number of rows in the font's dot-matrix template. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 7 rows.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of rows, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_SIZE_TEMPLATE +	Inquires the total number of dots possible in the font's dot-matrix template. The returned value corresponds to M_FONT_SIZE_COLUMNS * M_FONT_SIZE_ROWS.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of dots.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CHARS +	Inquires the number of characters in the font. Characters are added with MdmrImportFont() or MdmrControlStringModel().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of characters.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_FONT_SIZE_COLUMNS; M_FONT_SIZE_ROWS.

You can add the following value to the above-mentioned values to get the default value of an inquire type (if supported), regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT	Inquires the default value of the specified inquire type.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a font's characters, set the InquireType parameter to one of the values below. Unless otherwise specified, the FontLabelOrIndex parameter must be set to the label or index of a font, and either the CharIndex or CharNamePtr parameter must specify a character.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a font's characters																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CHAR_INDEX_VALUE +	Inquires the index value of the character, based on its name. Set the CharIndex parameter to M_DEFAULT and the CharNamePtr parameter to the name of the character in the font.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the character to which you are referring does not exist in the font.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the character.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NAME +	Inquires the name of the character, based on its index. Set the CharIndex parameter to the index of a character in the font and the CharNamePtr parameter to M_NULL. M_CHAR_NAME is returned as a regular string. It consists of all characters including the terminating null character.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the name.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_TEMPLATE +	Inquires the dot-matrix that represents the character in the font. The dot-matrix is returned as array data, organized one row contiguously after the other, where 255 (0xFF) represents a dot and 0 represents a blank. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the character's dot-matrix.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CHAR_TEMPLATE.

You can add the following value to the above-mentioned value to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CHAR_NAME.

You can add the following value to the above-mentioned value to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the size of a string																	
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE +	Inquires the number of characters in the string. This number accounts for every character, including the terminating null character. All strings have the terminating null character, even though you need not explicitly list it when specifying the string.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_CHAR_NAME; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to determine the hexadecimal 16-bit Unicode (UTF-16) character name information.

The character name of the letter A is "A" (its native name) or "\x0041", which is its representation in hexadecimal format beginning with "\x". Characters between 0 and 127 are considered Basic Latin. A character such as the smiley face (☺, or "\x263A" in hexadecimal) is beyond the Basic Latin range.

In a Unicode environment, the character representations are returned (for example, the smiley face), unless you explicitly specify one of the below combination values (for example, to obtain "\x263A" instead of the actual smiley face). Retrieving information about character names in hexadecimal might be useful if you want to convert your data into ASCII.

Note that the string size information can change when these combination values are added. For example, if the first four characters in the font are "abc☺", using M_CHAR_NAME + M_HEX_UTF16_FOR_NON_BASIC_LATIN returns "b" for the character at index 1, and "\x263A" for the character at index 3. Consequently, the string sizes are 2 and 7, respectively (inquired with M_CHAR_NAME + M_STRING_SIZE + M_HEX_UTF16_FOR_NON_BASIC_LATIN). If M_HEX_UTF16_FOR_ALL is specified, the preceding example returns "\x0062" and "\x263A" for the respective character names, and 7 for each string size. Recall that the null-terminating character is included in the string size.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For explicitly inquiring about hexadecimal 16-bit Unicode (UTF-16) character name information																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HEX_UTF16_FOR_ALL	Retrieves results with the name of all characters in hexadecimal 16-bit Unicode (UTF-16) format.	c		e	f	g	a		i		j	k	l	m	n	b
M_HEX_UTF16_FOR_NON_BASIC_LATIN	Retrieves results with the names of characters that fall out of the Basic Latin range, in hexadecimal 16-bit Unicode (UTF-16) format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a global setting of a font, For inquiring about a font's characters.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported for the font currently being inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring if an inquire type is supported																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a global setting of a font; and for the following values: M_CHAR_INDEX_VALUE; M_CHAR_TEMPLATE; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the requested information to the required data type

InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_UINT8	Casts the requested information to an *AIL_UINT8*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT8*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrInquireStringModel

Synopsis

Inquire about a global setting of a SureDotOCR string model, or inquire about a position (constraint) in a string model.

Syntax

```
AIL_INT MdmrInquireStringModel(
    AIL_ID ContextDmrId,               //in
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 Position,                //in
    AIL_INT64 PermittedCharEntry,      //in
    AIL_INT64 InquireType,             //in
    void *UserVarPtr                   //out
)
```

Description

This function inquires about a global setting of a SureDotOCR string model, or inquires about the different positions (constraints) in a string model. String models are defined with MdmrControlStringModel().

To inquire about the default constraints for the positions in the string model, call this function and pass M_DEFAULT to the Position parameter; to inquire about explicit constraints for a position, pass M_POSITION_IN_STRING(n) instead, where *n* is the explicitly constrained position. Note that constraints typically refer to the characters that are permitted to be read, according to a character type and font.

If you want to loop through all explicitly constrained positions, you can set the Position parameter to M_POSITION_CONSTRAINED_ORDER(n), where *n* is the order in which the position was explicitly constrained. Note that the constrained order of a position changes if you reset previously constrained positions to be implicitly constrained, using MdmrControlStringModel() with M_RESET_POSITION_TO_IMPLICIT_CONSTRAINTS.

If the inquired setting is set to M_DEFAULT (for example, in MdmrControlStringModel()), **MdmrInquireStringModel**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context that contains the string model about which to inquire. The context must have been previously allocated on the system using MdmrAlloc().

StringModelLabelOrIndex

Specifies the string model about which to inquire. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to inquire about the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the string model.															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies to inquire about the string model by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Specifies the label of the string model.															

			0 < *Value* < 2097152	Specifies the label of the string model about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

Position

Specifies how to inquire about the string model. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to inquire about the string model																		
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to inquire about a global setting of a string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_CONSTRAINED_ORDER(AIL_INT OrderValue)	Specifies to inquire about an explicitly constrained position in the string model by indicating the order in which the position was explicitly constrained.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	OrderValue Specifies the order in which the position was explicitly constrained.															
	0 <= *Value* <= 255 Specifies the order in which the position was explicitly constrained. For example, if you explicitly constrained positions 19, 5, and 24, their respective constrained order values are 0, 1, and 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_IN_STRING(AIL_INT PositionValue)	Specifies to inquire about an explicitly constrained position in the string model by indicating its position. If you specify a position that is not explicitly constrained, you will get an error. To inquire about the default constraints for the positions in the string model, you must set this parameter to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PositionValue Specifies the explicitly constrained position.															
	0 <= *Value* <= 255 Specifies the explicitly constrained position about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

PermittedCharEntry

Specifies the permitted character about which to inquire. SureDotOCR keeps track of each permitted character that you specify as an entry (MdmrControlStringModel() with M_ADD_PERMITTED_CHARS_ENTRY). The PermittedCharEntry parameter allows you to inquire about a specific entry.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the permitted character																		
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that this parameter is not required.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the permitted character about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of inquire to perform.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MdmrInquireStringModel**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a global setting of a string model
- For inquiring about constraints for the different positions in the string model

To inquire about a global setting of a string model, set the InquireType parameter to one of the values below. Unless otherwise specified, set the StringModelLabelOrIndex parameter to the label or index of a string model, set the Position parameter to M_DEFAULT, and set the PermittedCharEntry parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a global setting of a string model																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CHAR_ACCEPTANCE +	Inquires the acceptance level for the score of the string's characters. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the acceptance level for the character score, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CONSTRAINED_POSITIONS +	Inquires the number of explicitly constrained positions. These are set with MdmrControlStringModel().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of explicitly constrained positions.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_ACCEPTANCE +	Inquires the acceptance level for the string's score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the acceptance level for the string's score, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CERTAINTY +	Inquires the certainty level for the string's score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 70.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the certainty level for the string score, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX_VALUE +		Inquires the index of the string model. Set the StringModelLabelOrIndex parameter to the string model's label.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the string model you are referring to does not exist in the context.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL_VALUE +		Inquires the label of the string model. Set the StringModelLabelOrIndex parameter to the string model's index.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_RANK +		Inquires the order in which to read a string, relative to the other strings to read. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the order in which to read a string. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX +		Inquires the maximum number of characters in the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies an invalid maximum number of characters. Set the maximum number of characters to a valid value, using MdmrControlStringModel() with M_STRING_SIZE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 256	Specifies the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN +		Inquires the minimum number of characters in the string. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 256	Specifies the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about constraints for the different positions in the string model, set the InquireType parameter to one of the values below. In this case, set the StringModelLabelOrIndex parameter to the label or index of a string model.

To inquire about the default constraints for the positions in the string model, set the Position parameter to M_DEFAULT, unless otherwise specified. To inquire about explicit constraints for a position, set the Position parameter to the required position, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about constraints for the different positions in the string model																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_CHAR_LIST +		Inquires the names of the characters that can be read. This is returned as a regular string. It consists of all characters including the terminating null character. Characters can come from one or any font (M_FONT_LABEL_VALUE). M_CHAR_LIST is set as a permitted character constraint (M_ADD_PERMITTED_CHARS_ENTRY). There can be many permitted character constraint entries for a single character position. To specify which to inquire about, use the PermittedCharEntry parameter. SET														
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
	CharName	Specifies the character names. Retrieving an empty string ("") indicates that any character can be read.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINED_ORDER +		Inquires the order in which the position was explicitly constrained. Set the Position parameter to M_POSITION_IN_STRING().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the constrained order. This value will be less than M_NUMBER_OF_CONSTRAINED_POSITIONS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL_VALUE +		Inquires the label of the font from which the character to read must come. This is set as a permitted character constraint (M_ADD_PERMITTED_CHARS_ENTRY). There can be many such permitted character constraint entries (allowable fonts) for a single character position. To specify which to inquire about, use the PermittedCharEntry parameter. To inquire the corresponding font index, call MdmrInquireFont() with M_FONT_INDEX_VALUE and the label value returned by M_FONT_LABEL_VALUE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANY	Specifies that the character can come from any font in the context.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of the font. If SureDotOCR cannot return the requested information (no such font exists in the context), you will get an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_OPTIONAL +		Inquires whether a position in a string model is optional. Optional positions can be skipped if their constraints cannot be met (up to a maximum of M_STRING_SIZE_MAX- M_STRING_SIZE_MIN skips). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the model position is not optional.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the model position is optional.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_PERMITTED_CHARS_ENTRIES +		Inquires the number of permitted character entries. This is the number of permitted characters specified with M_ADD_PERMITTED_CHARS_ENTRY. Having no permitted character entries indicates you are reading any character from any font (initial default behavior).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of permitted character entries.	c		e	f	g	a		i		j	k	l	m	n	b

M_POSITION +	Inquires the position within the string model, corresponding to the order in which the position was explicitly constrained. Set the Position parameter to M_POSITION_CONSTRAINED_ORDER().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the position.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE +	Inquires the type of characters that can be read. This is set as a permitted character constraint (M_ADD_PERMITTED_CHARS_ENTRY). There can be many such allowable types of characters for a position. To specify the one about which to inquire, use the PermittedCharEntry parameter. If you are reading an explicitly defined list of characters, use M_CHAR_LIST to inquire the names of those characters. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies to read all characters present in the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_LIST	Specifies to read an explicit list of characters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGITS	Specifies to read characters '0' to '9'.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS	Specifies to read characters 'A' to 'Z' and 'a' to 'z'.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_LOWERCASE	Specifies to read characters 'a' to 'z'.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_UPPERCASE	Specifies to read characters 'A' to 'Z'.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACE	Specifies to read a space. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CHAR_LIST.

You can add the following value to the above-mentioned value to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the size of a string																
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE +	Inquires the number of characters in the string. This number accounts for every character, including the terminating null character. All strings have the terminating null character, even though you need not explicitly list it when specifying the string.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_CHAR_LIST; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to determine the hexadecimal 16-bit Unicode (UTF-16) character name information.

The character name of the letter A is "A" (its native name) or "\x0041", which is its representation in hexadecimal format beginning with "\x". Characters between 0 and 127 are considered Basic Latin. A character such as the smiley face (☺, or "\x263A" in hexadecimal) is beyond the Basic Latin range.

In a Unicode environment, the character representations are returned (for example, the smiley face), unless you explicitly specify one of the below combination values (for example, to obtain "\x263A" instead of the actual smiley face). Retrieving information about character names in hexadecimal might be useful if you want to convert your data into ASCII.

Note that the string size information can change when these combination values are added. For example, if the first four characters in the font are "abc☺", using M_CHAR_LIST + M_HEX_UTF16_FOR_NON_BASIC_LATIN returns "b" for the character at index 1, and "\x263A" for the character at index 3. Consequently, the string sizes are 2 and 7, respectively (inquired with M_CHAR_LIST + M_STRING_SIZE +

M_HEX_UTF16_FOR_NON_BASIC_LATIN). If M_HEX_UTF16_FOR_ALL is specified, the preceding example returns "\x0062" and "\x263A" for the respective character names, and 7 for each string size. Recall that the null-terminating character is included in the string size.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For explicitly inquiring about hexadecimal 16-bit Unicode (UTF-16) character name information																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HEX_UTF16_FOR_ALL	Retrieves results with the name of all characters in hexadecimal 16-bit Unicode (UTF-16) format.	c		e	f	g	a		i		j	k	l	m	n	b
M_HEX_UTF16_FOR_NON_BASIC_LATIN	Retrieves results with the names of characters that fall out of the Basic Latin range, in hexadecimal 16-bit Unicode (UTF-16) format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For inquiring about a global setting of a string model, For inquiring about constraints for the different positions in the string model. (exceptions might exist)

You can add the following value to the above-mentioned values to get the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_DEFAULT	Inquires the default value of the specified inquire type.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a global setting of a string model, For inquiring about constraints for the different positions in the string model.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported for the font currently being inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring if an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_HAS_DEFAULT	Inquires whether the specified inquire type has a default value.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED	Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a global setting of a string model; and for the following values: M_CONSTRAINED_ORDER; M_FONT_LABEL_VALUE; M_IS_OPTIONAL; M_NUMBER_OF_PERMITTED_CHARS_ENTRIES; M_POSITION; M_TYPE; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting information to the required data type.																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrName

Synopsis

Perform name operations for fonts and string models.

Syntax

```
void MdmrName(
    AIL_ID ContextDmrId,    //in
    AIL_INT64 Operation,    //in
    AIL_INT64 LabelOrIndex, //in
    AIL_TEXT_PTR Name,      //in-out
    AIL_INT *ValuePtr,      //out
    AIL_INT64 ControlFlag   //in
)
```

Description

This function allows you to perform name operations for fonts and string models in a SureDotOCR context. Use this function to set the name of a font or string model, release the name of a font or string model, or retrieve either the label or index of a font or string model based on its name. These operations allow you to more easily manage your data; they do not impact how strings are processed and read.

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context containing the font or string model with which to perform the name operation. The context must have been previously allocated on the system using MdmrAlloc().

Operation

Specifies the name operation to perform. Set this parameter to one of the values below. Unless otherwise specified, the LabelOrIndex parameter can specify a font or string model, the Name parameter must specify the name information related to the operation, and the ValuePtr parameter must specify the address at which to write the information requested by the name operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name operation																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GET_FONT_INDEX	Retrieves the index of the font, based on the specified name. Set the LabelOrIndex parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_FONT_LABEL	Retrieves the label of the font, based on the specified name. Set the LabelOrIndex parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_NAME	Retrieves the name, and name length, of the specified font or string model. Use the Name parameter to retrieve the name. Use the ValuePtr parameter to retrieve the length of the name.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_STRING_INDEX	Retrieves the index of the string model, based on the specified name. Set the LabelOrIndex parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_STRING_LABEL	Retrieves the label of the string model, based on the specified name. Set the LabelOrIndex parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_SET_NAME	Sets the name of the font or string model, or releases the name of the font or string model. When setting a name, ensure it is not already used. All font names must be unique among font names and all string model names must be unique among string model names. When releasing a name, set the Name parameter to **M_NULL**. Once released, you can use the name again.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies the label or index value of the font or string model with which to perform the name operation. Set this parameter to **M_DEFAULT** when not required by the name operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label or index value																		
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FONT_INDEX(AIL_INT IndexValue)	Specifies a font by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_FONTS — Specifies the index of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_FONT_LABEL(AIL_INT LabelValue)	Specifies a font by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Set this parameter to the following:															
	Value > 0 — Specifies the label of the font.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX(AIL_INT IndexValue)	Specifies a string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Set this parameter to the following:															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS — Specifies the index of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies a string model by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Set this parameter to the following:															
	Value > 0 — Specifies the label of the string model.	c		e	f	g	a		i		j	k	l	m	n	b

Name

Specifies the name. Use this parameter to set, get, or release the name, depending on the name operation. If the operation is getting a label and it fails, SureDotOCR returns 0. If the operation is getting an index and it fails, SureDotOCR returns -1. To release the name of a font or string model, set this parameter to **M_NULL** and specify M_SET_NAME. To determine the length of the current name of the font or string model, set this parameter to **M_NULL** and specify M_GET_NAME.

ValuePtr

Specifies the address at which to write the information requested by the name operation.

Set this parameter to **M_NULL** when not required by the name operation.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrPreprocess

Synopsis

Preprocess a SureDotOCR context.

Syntax

```
void MdmrPreprocess(
    AIL_ID ContextDmrId,  //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function prepares a SureDotOCR context for a read operation. This allows SureDotOCR to make internal refinements so it can execute optimized and robust dot-matrix character recognition processes. You must call this function before the first call to MdmrRead(). To preprocess a context without error, it must contain at least one font, one character, and one string model.

Changes to a context or to any of its content often require you to preprocess the context again. To inquire a context's preprocessing state, call MdmrInquire() with M_PREPROCESSED.

Saving a context does not save preprocessing changes. Upon restoration, you must preprocess the context again.

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context to preprocess. The context must have been previously allocated on the system using MdmrAlloc().

ControlFlag

Specifies whether to preprocess the SureDotOCR context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to preprocess the context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Preprocesses the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET	Un-preprocesses the context. Un-preprocessing the context can be useful if you want to conserve system memory within an application and preserve context settings.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrRead

Synopsis

Read dot-matrix strings from a target image.

Syntax

```
void MdmrRead(
    AIL_ID ContextDmrId,     //in
    AIL_ID TargetImageBufId, //in
    AIL_ID ResultDmrId,      //out
    AIL_INT64 ControlFlag    //in
)
```

Description

This function uses a SureDotOCR context to read one or more dot-matrix strings from a target image. Results are stored in a SureDotOCR result buffer. To get the results, use MdmrGetResult().

Requirements for reading strings depend on the settings of the context, and the font and string models therein. To control these settings, use MdmrControl(), MdmrControlFont(), and MdmrControlStringModel(). **MdmrRead**() takes into account all settings.

Before performing a read operation, there must be at least one font, one character, and one string model in the context; it must also be in a preprocessed state (MdmrPreprocess()).

To read a string, it must not only meet the context's requirements, but it must also follow the basic rules of a string. It must, for example, be a linear sequence of characters.

Strings are read in the natural Latin-based reading order; that is, from left to right, and from top to bottom (with respect to the angle of the target). A string's position is the position of the center of the bounding box of its first character.

You can limit the read operation to a region of the target image buffer, using a rectangular ROI set with MbufSetRegion().

If the target image is not calibrated, results are calculated in the pixel coordinate system (pixel units). If the target image is calibrated, results are calculated in the world coordinate system (real-world units), but you can retrieve them in either world or pixel units. To specify this, call MdmrControl() with M_RESULT_OUTPUT_UNITS.

In the presence of distortion, some results are meaningless when converted from real-world to pixel units (such as angle and scale). For example, if a string appears warped in the target image, but the camera calibration context compensates for this, the resulting angle is meaningful in the real-world coordinate system, and meaningless in the pixel coordinate system. If complex distortions exist, correct the image before using it with SureDotOCR.

The read operation can occasionally take an unexpectedly long time to calculate. Use MdmrControl() with M_TIMEOUT to set a maximum read time.

Parameters

ContextDmrId

Specifies the identifier of the SureDotOCR context to use for the read operation. The context must have been previously allocated on the required system using MdmrAlloc(), and preprocessed using MdmrPreprocess().

TargetImageBufId

Specifies the identifier of the target image buffer which contains the dot-matrix strings to read. The target image buffer must be a 1-band, 8-bit unsigned buffer. Its maximum size is 65536x65536 pixels.

This image buffer can only have a rectangular region of interest (ROI) defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

ResultDmrId

Specifies the identifier of the SureDotOCR result buffer in which to write the results of the read operation. The SureDotOCR result buffer must have been previously allocated on the required system using MdmrAllocResult().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrRestore

Synopsis

Restore a SureDotOCR context from disk.

Syntax

```
AIL_ID MdmrRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextDmrIdPtr      //out
)
```

Description

This function restores a SureDotOCR context that was previously saved to a file, using MdmrSave() or MdmrStream(). This function restores all of the context's settings (including font and string model settings) that were in effect when the context was saved. A restored context is not preprocessed; therefore, you must call MdmrPreprocess() before performing a read with MdmrRead().

When the restored SureDotOCR context is no longer required, release it using MdmrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored SureDotOCR context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the SureDotOCR context. The function handles (internally) the opening and closing of the file. For easier use with other Aurora Imaging Library software products, files for SureDotOCR contexts should use the MDMR file extension. Predefined SureDotOCR context files are typically distributed with Aurora Imaging Library and can be found in the installed Contexts folder (for example, *C:\Program Files\Aurora Imaging Library\11\Contexts*").

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the SureDotOCR context file (for example, "*C:\mydirectory\MyContext.mdmr*"). Typically, SureDotOCR context files have an MDMR extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdmr*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the identifier of the system on which to restore the SureDotOCR context.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextDmrIdPtr

Specifies the address of the variable in which to write the SureDotOCR context identifier or specifies the data type that the function should use to return the SureDotOCR context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated SureDotOCR context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated SureDotOCR context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DMR_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the SureDotOCR context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated SureDot OCR context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the SureDotOCR context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DMR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrSave

Synopsis

Save a SureDotOCR context to a file.

Syntax

```
void MdmrSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextDmrId,         //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated SureDotOCR context to disk, including all of the individual font and string model settings. This information can be reloaded, using MdmrRestore() or MdmrStream().

When you save the context, the preprocessing changes are not saved. Upon restoration, you must preprocess the context again.

Parameters

FileName

Specifies the name and path of the file in which to save the SureDotOCR context. The function internally handles the opening and closing of this file. For easier use with other Aurora Imaging Library software products, files for SureDotOCR contexts should use the MDMR file extension. Predefined SureDotOCR context files are typically distributed with Aurora Imaging Library and can be found in the installed Contexts folder (for example, "*C:\Program Files\Aurora Imaging Library\11\Contexts*"). If you specify a file that already exists, it will be overwritten.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the SureDotOCR context file (for example, "*C:\mydirectory\MyContext.mdmr*"). Typically, SureDotOCR context files have an MDMR extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdmr*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextDmrId

Specifies the identifier of the SureDotOCR context to save.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.

MdmrStream

Synopsis

Load, restore, or save a SureDotOCR context from/to a file or a memory stream.

Syntax

```
void MdmrStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextDmrIdPtr,       //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load or restore a SureDotOCR context from a file or memory stream. All of the context settings that were in effect when the context was saved will be restored. A loaded or restored context is not preprocessed. You must call MdmrPreprocess() before performing a read with MdmrRead().

This function can also save a SureDotOCR context to a file or memory stream. All information about the previously allocated context is saved, including all of the individual font and string model settings. Preprocessing changes are not saved.

To inquire the number of bytes necessary to save a SureDotOCR context to memory stream, first call **MdmrStream**() with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with MdmrSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

Using **MdmrStream**(), you can choose to save a backwards-compatible version of the SureDotOCR context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore SureDotOCR contexts saved using Aurora Imaging Library version 10.0 or above. Settings that do not exist in the lower version will be filled with default values when the SureDotOCR context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the identifier of the system on which to restore the SureDotOCR context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the SureDotOCR context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the SureDotOCR context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the SureDotOCR context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextDmrIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the SureDotOCR context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the SureDotOCR context, in bytes. If size is not required, set this parameter to **M_NULL**.

> The size of the context varies depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextDmrIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextDmrIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a SureDotOCR context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the SureDotOCR context must be compatible when saved. The size of a context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b

M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDmrIdPtr	Specifies the address of the variable containing the identifier of the SureDotOCR context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated SureDotOCR context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\MyContext.mdmr*"), when the StreamType parameter is set to M_FILE. SureDotOCR contexts typically have an MDMR file extension. The function handles (internally) the opening and closing of the file. Predefined SureDotOCR context files are typically distributed with Aurora Imaging Library and can be found in a local folder (for example, "C:\Program Files\Aurora Imaging Library\<version number>\Library\Contexts\"). To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdmr*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDmrIdPtr	Specifies the address of the variable containing the identifier of the SureDotOCR context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores a SureDotOCR context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\MyContext.mdmr*"), when the StreamType parameter is set to M_FILE. SureDotOCR contexts typically have an MDMR file extension. The function handles (internally) the opening and closing of the file. Predefined SureDotOCR context files are typically distributed with Aurora Imaging Library and can be found in a local folder (for example, "C:\Program Files\Aurora Imaging Library\<version number>\Library\Contexts\"). To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdmr*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the SureDotOCR context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDmrIdPtr	Specifies the address of the variable in which to write the identifier of the SureDotOCR context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a SureDotOCR context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\MyContext.mdmr*"), when the StreamType parameter is set to M_FILE. SureDotOCR contexts typically have an MDMR file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. Predefined SureDotOCR context files are typically distributed with Aurora Imaging Library and can be found in a local folder (for example, "C:\Program Files\Aurora Imaging Library\<version number>\Library\Contexts\"). To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdmr*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the SureDotOCR context must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextDmrIdPtr		Specifies the address of the variable containing the identifier of the SureDotOCR context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildmr11.lib.
DLL	Requires ail11.dll; aildmr11.dll.